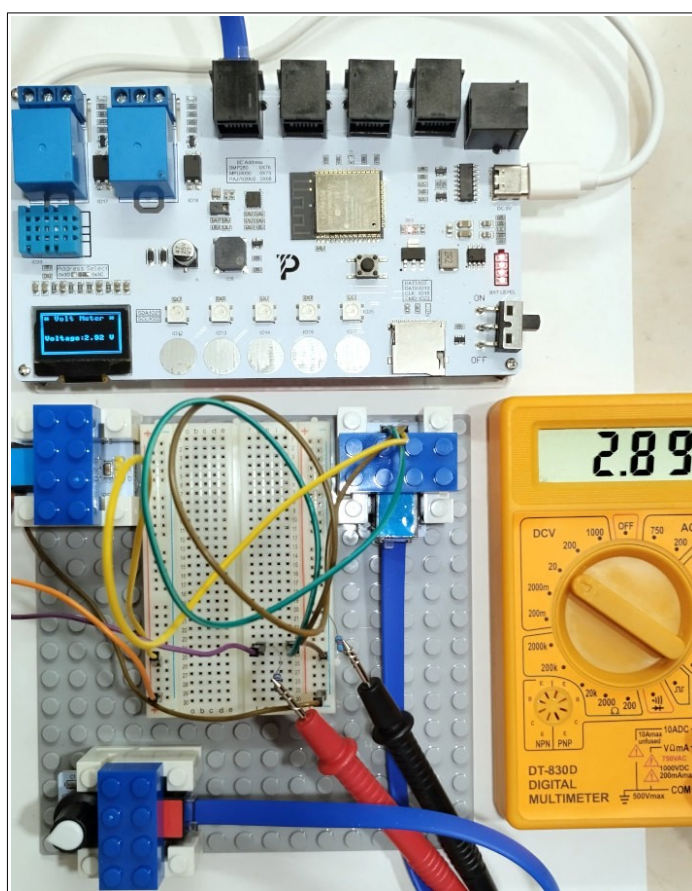


Άσκησης Micropython

στο σύστημα ARD:Icon II της Polytech



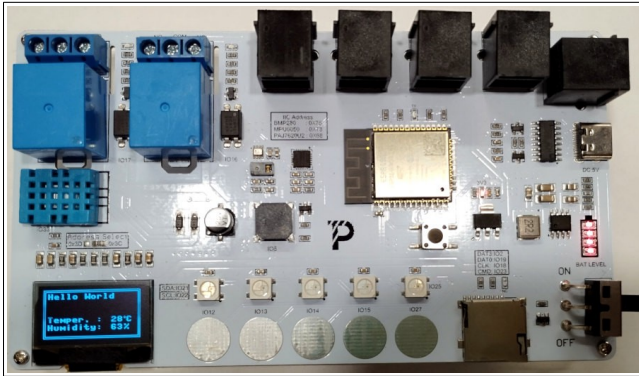
Έκδοση 1.1

Σταύρος Σ. Φώτογλου ΠΕ86
Ε.Κ. Πρέβεζας – 1ο ΕΠΑ.Λ. Πρέβεζας

Πρέβεζα 2025-26

Άσκηση 1

Στην πλακέτα ARD:icon II (IOT) του kit ρομποτικής S4T της Polytech, να γίνει πρόγραμμα σε Micropython το οποίο εμφανίζει την θερμοκρασία και την σχετική υγρασία από τον αισθητήρα DHT 11 στην ενσωματωμένη οθόνη OLED του συστήματος. Οι τιμές θα ανανεώνονται κάθε 2 δευτερόλεπτα.



Μέσα στο σύστημα αρχείων πρέπει να αντιγραφτεί η βιβλιοθήκη `ssd1306.py`

```

1 #Πρόγραμμα το οποίο παρουσιάζει μετρήσεις Θερμοκρασίας και υγρασίας από DHT11 στην οθόνη OLED
2 import machine, time, dht
3 i2c = machine.I2C(0, sda = machine.Pin(21), scl = machine.Pin(22), freq = 400000)
4 #Η εναλλακτικά
5 #i2c = machine.SoftI2C(sda=machine.Pin(21), scl=machine.Pin(22))
6 from ssd1306 import SSD1306_I2C
7 sensor = dht.DHT11(machine.Pin(33))
8 sensor.measure()
9
10 #display = SSD1306_I2C(128, 64, i2c) #Δίνω μπορώ να βάλω διεύθυνση π.χ. SSD1306_I2C(128, 64, i2c, 0x3c)
11 display = SSD1306_I2C( width=128, height=64, i2c=i2c, addr=0x3c, external_vcc=False )
12 display.fill(0) #Καθαρίζει οθόνη
13 #Δημιουργώ ετικέτες
14 display.text('Hello World', 5, 5, 1) #Θέση x, y, color 1/0
15 display.text("Temper. : ", 5, 35, 1)
16 display.ellipse(106, 36, 1, 1, 1)
17 display.text("C", 108, 35, 1)
18 display.text("Humidity: ", 5, 45, 1)
19 display.text("%", 105, 45, 1)
20 #Δημιουργώ διπλό περίγραμμα
21 display.rect(0,0,127,63,1,0) #x, y, width, height, color=0/1, fill=0/1
22 display.rect(2,2,123,59,1,0) #x, y, width, height, color=0/1, fill=0/1
23 display.show() #Παρουσιάζει στην οθόνη
24
25 #Για πάντα
26 while True:
27     sensor.measure()
28     display.rect(80, 34, 25, 9, 0, True) #Σβήσε παλιά τιμή
29     display.text("{: 3}".format(sensor.temperature()), 80, 35, 1) #Εμφάνισε νέα τιμή
30     display.rect(80, 44, 25, 9, 0, True) #Σβήσε παλιά τιμή
31     display.text("{: 3}".format(sensor.humidity()), 80, 45, 1) #Εμφάνισε νέα τιμή
32     print("Θερμοκρασία:", sensor.temperature()) #Εμφάνισε στο τερματικό
33     print("Σχετ. Υγρασία:", sensor.humidity())
34     display.show() #Παρουσιάζει στην οθόνη
35     time.sleep(2) #Περίμενε 2 sec
    
```

Άσκηση 2

Να τροποποιηθεί το προηγούμενο ώστε να εμφανίζει και μετρήσεις του αισθητήρα BMP280. Ο αισθητήρας BMP280 μπορεί να μετρήσει ατμοσφαιρική πίεση και θερμοκρασία.

Μέσα στο σύστημα αρχείων πρέπει να αντιγραφτεί η βιβλιοθήκη `ssd1306.py` και η βιβλιοθήκη `bmp280.py`.

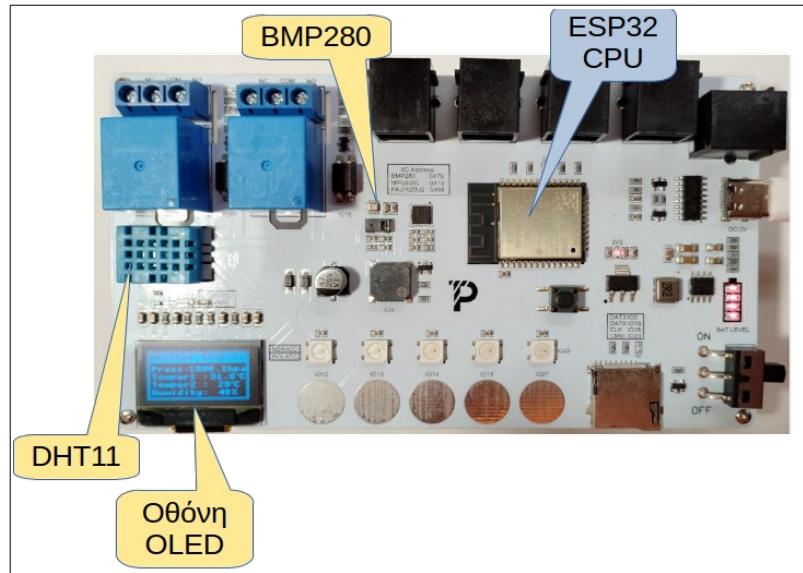
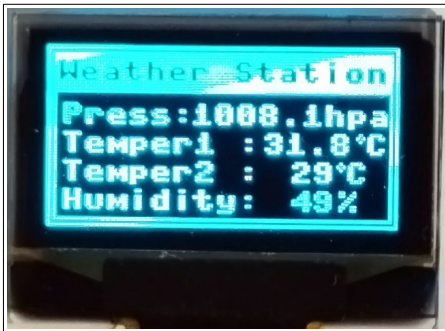
```

1 # Πρόγραμμα το οποίο παρουσιάζει μετρήσεις Θερμοκρασίας και υγρασίας από DHT11
2 # και ατμ. πίεσης και θερμοκρασίας από το BMP280 στην οθόνη OLED
3 import machine, time, dht, math
4 i2c = machine.I2C(0, sda = machine.Pin(21), scl = machine.Pin(22), freq = 400000)
5 #Η εναλλακτικά
6 #i2c = machine.SoftI2C(sda=machine.Pin(21), scl=machine.Pin(22))
7 from ssd1306 import SSD1306_I2C #0 ελεγκτής της οθόνης OLED
8 sensor = dht.DHT11(machine.Pin(33)) #0 αισθητήρας DHT11
9 from bmp280 import *
10 bmp = BMP280(i2c) #Στιγμιότυπο του BMP280
11 #Στιγμιότυπο της οθόνης
12 display = SSD1306_I2C(width=128, height=64, i2c=i2c, addr=0x3c, external_vcc=False)
13 display.fill(0) #Καθαρίζει οθόνη
14
15 #Δημιουργώ διπλό περίγραμμα
16 display.rect(0, 0, 127, 63, 1, 0) #x, y, width, height, color=0/1, fill=0/1
17 display.rect(2, 2, 123, 59, 1, 0) #x, y, width, height, color=0/1, fill=0/1
18
    
```

```

19 #Δημιουργίες ετικέτες
20 display.rect(3, 2, 121, 14, 1, 1) #x, y, width, height, color=0/1, fill=0/1
21 display.text('Weather Station', 3, 5, 0) #θέση x, y, color 1/0
22 display.text("Press: ", 5, 20, 1)
23 display.text("hpa", 100, 20, 1)
24 display.text("Temper1 : ", 5, 30, 1)
25 display.ellipse(113, 31, 1, 1, 1)
26 display.text("C", 115, 30, 1)
27 display.text("Temper2 : ", 5, 40, 1)
28 display.ellipse(106, 41, 1, 1, 1)
29 display.text("C", 108, 40, 1)
30 display.text("Humidity: ", 5, 50, 1)
31 display.text("%", 105, 50, 1)
32 display.show() #Παρουσίασε στην οθόνη
33
34 #Για πάντα
35 while True:
36     bmp.force_measure() #Διάβασε τιμές από BMP280
37     display.rect(52, 19, 48, 9, 0, True) #Εβήσε παλιά τιμή ατμ. πίεσης
38     display.text(str(round(bmp.pressure / 100), 1)), 52, 20, 1) #Εμφάνισε νέα τιμή ατμ. πίεσης
39     display.rect(77, 29, 34, 9, 0, True) #Εβήσε παλιά τιμή θερμοκρασίας BMP
40     display.text(str(round(bmp.temperature, 1)), 78, 30, 1) #Εμφάνισε νέα τιμή θερμοκρασίας BMP
41     print("Θερμοκρασία BMP: " + str(round(bmp.temperature, 1))) #Εμφάνισε σε τερματικό
42     print("Πίεση: " + str(round(bmp.pressure / 100), 1)) #Εμφάνισε σε τερματικό
43
44     sensor.measure() #Διάβασε τιμές από DHT11
45     display.rect(80, 39, 25, 9, 0, True) #Εβήσε παλιά τιμή Θερμοκρασίας DHT
46     display.text("{: 3}".format(sensor.temperature()), 80, 40, 1) #Εμφάνισε νέα τιμή θερμοκρ. DHT
47     display.rect(80, 49, 25, 9, 0, True) #Εβήσε παλιά τιμή σχετ. υγρασίας
48     display.text("{: 3}".format(sensor.humidity()), 80, 50, 1) #Εμφάνισε νέα τιμή υγρασίας
49     print("Θερμοκρασία DHT:", sensor.temperature()) #Εμφάνισε στο τερματικό
50     print("Σχετ. Υγρασία:", sensor.humidity()) #Εμφάνισε στο τερματικό
51     print() #Άσε μια γραμμή κενή
52
53     display.show() #Παρουσίασε στην οθόνη
54     time.sleep(2) #Περίμενε 2 sec

```



Ασκηση 3

Να γίνει πρόγραμμα σε Micropython το οποίο θα ανιχνεύει το άγγιγμα των κουμπιών IO12 και IO13. Όταν πατιέται το IO12 θα ανάβει το RGB Led πάνω απ' αυτό σε λευκό χρώμα και όταν πατιέται το IO13 θα σβήνει. Κατά το πάτημα των κουμπιών θα ακούγεται και χαρακτηριστικός ήχος από τον βομβητή.

Για την υλοποίηση δεν χρειάζονται επιπλέον βιβλιοθήκες.

```

1 import machine, time, neopixel
2
3 #Ορίσμοι
4 threshold = 200 # Threshold to be adjusted
5 Debounce = 50 #msec
6
7 #Στιγμιότυπο RGB Leds
8 rgb = neopixel.NeoPixel(machine.Pin(25), 5) #IO25, 5 x RGB Leds
9
10 #Ορίσμος ακροδεκτών κουμπιών αφής
11 t_pin1 = machine.TouchPad(machine.Pin(12)) #1ο κουμπί αφής
12 t_pin2 = machine.TouchPad(machine.Pin(13)) #2ο κουμπί αφής

```

```

13
14 #Ορισμός ακροδέκτη Beeper
15 beeper = machine.Pin(5, machine.Pin.OUT) #Ο βομβητής συνδέεται στο pin IO5
16 #Συνάρτηση παραγωγής τόνου συχνότητας freq σε Hz και διάρκειας dur σε msecs
17 def beep(freq = 500, dur = 500): #Προκαθορισμένο 500Hz, 500msec
18     #Υπολογισμοί
19     period = 1.0 / freq #Περίοδος
20     half_per = int((period / 2) * 1000000) #Ημιπερίοδος σε msecs
21     times = int(dur / 1000 / period) #Αριθμός κύκλων
22     for i in range(times):
23         beeper.value(1)
24         time.sleep_us(half_per)
25         beeper.value(0)
26         time.sleep_us(half_per)
27
28 #----- Ορισμός κλάσης ελέγχου κουμπιών αφής (Μόνο απλό κλικ) -----
29 class Touch:
30     #Κατασκευαστής
31     def __init__(self, pin):
32         self.pin = pin #Το GPIO του κουμπιού αφής
33         self.validclick = False #Έγκυρο πάτημα
34         self.timer = 0 #Ο timer που κρατάει τα ticks σε msec
35         self.downtime = 0 #Πόσο χρόνο είναι πατημένο (Δάχτυλο πάνω στο κουμπί αφής)
36
37     #Συμβάν απλό κλικ
38     def onClick(self):
39         self()
40
41     #Καλείται περιοδικά και ελέγχει το κάθε κουμπί
42     def checkButton(self):
43         self.timer = time.ticks_ms() #Κράτησε τον χρόνο συστήματος
44         cVal = self.pin.read() #Διάβασε τιμή του αισθητήρα touch
45         state = False #Δεν πατήθηκε
46         if cVal < threshold: #Αν είναι κάτω από το κατώφλι (Δάχτυλο πάνω στην νησίδα)
47             state = True #Πατήθηκε το κουμπί
48         #----- Πατημένο -----
49         #Πατήθηκε τώρα για πρώτη φορά και ο χρόνος δεν μετράει
50         if state and self.downtime == 0:
51             self.downtime = self.timer #Αρχισε να μετράς τον χρόνο που είναι πατημένο - downtime
52         #Παραμένει πατημένο και έλεγξε αν πέρασε ο χρόνος debounce και δεν έχει ενεργοποιηθεί valid click
53         elif state and not self.validclick and (self.timer - self.downtime) > Debounce:
54             self.validclick = True #Να μην ξαναμπείς εδώ
55             self.onClick() #Κάλεσε συνάρτηση εξυπηρέτησης
56         #----- Ελεύθερο -----
57         #Αλλιώς αν ήταν πατημένο και τώρα το άφησε
58         elif not state:
59             self.downtime = 0 #Επαναφορά
60             self.validclick = False #Επαναφορά για την επόμενη φορά
61         #----- Τέλος ορισμού κλάσης -----
62
63 #----- Συναρτήσεις εξυπηρέτησης συμβάντων -----
64 def key1_click(): #Αναψε
65     print("Led is ON")
66     #----- R, G, B -----
67     rgb[4] = (255, 255, 255) #Τιμές από 0 (σβηστό) έως 255 (μέγιστη ένταση)
68     rgb.write()
69     beep(2000, 50)
70
71 def key2_click(): #Σβήσε
72     print("Led is OFF")
73     rgb[4] = (0, 0, 0)
74     rgb.write()
75     beep(400, 50)
76
77 #===== Κυρίως πρόγραμμα =====
78 #----- Δημιουργία στιγμιοτύπων και ορισμός συναρτήσεων εξυπηρέτησης -----
79 ts1 = Touch(t_pin1)
80 ts1.onClick = key1_click
81
82 ts2 = Touch(t_pin2)
83 ts2.onClick = key2_click
84
85 print("\nESP32 Touch Buttons & RGB Leds Demo")
86
87 #----- Για πάντα -----
88 while True:
89     ts1.checkButton() #Έλεγχος 1ου κουμπιού
90     ts2.checkButton() #Έλεγχος 2ου κουμπιού
91     time.sleep_ms(5) #Περίμενε 5 - 10msec

```

Άσκηση 4

Να κατασκευαστεί κλάση για τα κουμπιά αφής της πλακέτας η οποία ανιχνεύει μονό κλικ, διπλό κλικ και παρατεταμένο κλικ. Για κάθε συμβάν καλείται ξεχωριστή συνάρτηση εξυπηρέτησης. Επίσης να γραφεί κώδικας ο οποίος δημιουργεί στιγμιότυπα για δύο κουμπιά και εξυπηρετεί και τις τρεις λειτουργίες για το καθένα.

```

1 import machine, time
2 #----- Ορισμοί -----
3 threshold = 200 # Threshold to be adjusted
4 Debounce = 50 #msec
5 DbClickDelay = 300 #msec
6 LongPressDelay = 1000 #msec
7 t_pin1 = machine.TouchPad(machine.Pin(12)) #1ο κουμπί αφής
8 t_pin2 = machine.TouchPad(machine.Pin(13)) #2ο κουμπί αφής
9
10 #----- Ορισμός κλάσης -----
11 class Touch:
12     #Κατασκευαστής
13     def __init__(self, pin):
14         self.pin = pin
15         self.laststate = True
16         self.timer = 0
17         self.downtime = -1

```



```

18     self.uptime = -1
19     self.ignoreUP = False
20     self.singleClickOK = False
21     self.dblClickWaiting = False
22     self.dblClickOnNextUp = False
23     self.longPressHappened = False
24
25     #Συμβάντα
26     #Απλό κλικ
27     def onClick(self):
28         self()
29
30     #Διπλό κλικ
31     def onDblClick(self):
32         self()
33
34     #Παρατεταμένο κλικ
35     def onLongClick(self):
36         self()
37
38     #Καλείται περιοδικά και ελέγχει το κάθε κουμπί
39     def checkButton(self):
40         resultEvent = 0
41         self.timer = time.ticks_ms() #Κράτησε τον χρόνο συστήματος
42         cVal = self.pin.read() #Ανάβασε τιμή του αισθητήρα touch
43         state = False #Δεν πατήθηκε
44         if cVal < threshold: #Αν είναι κάτω από το κατώφλι
45             state = True #Πατήθηκε το κουμπί
46             #----- Πατημένο -----
47             #Πατήθηκε τώρα και πριν δεν είχε πατηθεί και ο uptime έχει ξεπεράσει τον χρόνο debounce.
48             #Άραικά θα μπει εδώ αμέσως όταν πατηθεί γιατί δεν έχει κρατηθεί ο uptime
49             if state == True and self.laststate == False and (self.timer - self.uptime) > Debounce:
50                 self.downtime = self.timer #Άρχισε να μετράς τον χρόνο που είναι πατημένο - downtime
51                 self.ignoreUP = False
52                 self.singleClickOK = True
53                 self.longPressHappened = False
54                 #Αν δεν έχει περάσει ο χρόνος uptime τον χρόνο double click και δεν έχει ενεργοποιηθεί το double click στην επόμενη επαναφορά
55                 #και περιμένει για double click. Εδώ θα μπει κατά το 2ο πάτημα
56                 if (self.timer - self.uptime) < DblClickDelay and self.dblClickOnNextUp == False and self.dblClickWaiting == True:
57                     self.dblClickOnNextUp = True #Στο άφημα να το θεωρήσει διπλό κλικ
58                 else: #Εδώ θα μπει στο 1ο κλικ του διπλού κλικ
59                     self.dblClickOnNextUp = False
60                     self.dblClickWaiting = False #Επανάφορά για την επόμενη φορά
61             #----- Ελεύθερο -----
62             #Αλλιώς αν ήταν πατημένο και τώρα το άφησε και πέρασε ο downtime έχει ξεπεράσει τον χρόνο debounce
63             #Εδώ θα μπει αν κρατήθηκε πατημένο για χρόνο > debounce
64             elif state == False and self.laststate == True and (self.timer - self.downtime) > Debounce:
65                 #Αν δεν θέλει να αγνοήσει το Up. Εδώ δεν μπαίνει κατά το άφημα του long click
66                 if self.ignoreUP == False:
67                     self.uptime = self.timer #Άρχισε να μετράς τον χρόνο που είναι πάνω - uptime χρήσιμος για double click
68                     if self.dblClickOnNextUp == False: #Έχει γίνει το 1ο κλικ και τώρα το άφησε
69                         self.dblClickWaiting = True #Θα περιμένει για δεύτερο πάτημα
70                     #Εδώ έχει συμβεί διπλό κλικ. Είναι πάνω μετά από διπλό κλικ
71                 else:
72                     resultEvent = 2 #Αποτέλεσμα διπλό κλικ
73                     self.dblClickOnNextUp = False #Επανάφορά
74                     self.dblClickWaiting = False
75                     self.singleClickOK = False
76
77             #Έλεγχος για μονό κλικ. Ο χρόνος του διπλού κλικ έχει λήξει
78             if state == False and (self.timer - self.uptime) >= DblClickDelay and self.dblClickWaiting == True and self.dblClickOnNextUp == False and \
79                 self.singleClickOK == True and resultEvent != 2:
80                 resultEvent = 1
81                 self.dblClickWaiting = False
82
83             #Έλεγχος για παρατεταμένο κλικ
84             if state == True and (self.timer - self.downtime) >= LongPressDelay:
85                 #Πυροδότησε το παρατεταμένο πάτημα
86                 #Αν δεν πατήθηκε πριν παρατεταμένα
87                 if self.longPressHappened == False:
88                     resultEvent = 3
89                     self.ignoreUP = True #Αγνόησε το άφημα
90                     self.dblClickOnNextUp = False
91                     self.dblClickWaiting = False
92                     self.longPressHappened = True
93
94             self.laststate = state #Κράτα την προηγούμενη κατάσταση
95             #Κλήσεις ανάλογα με την ενέργεια
96             if resultEvent == 1:
97                 self.onClick()
98             if resultEvent == 2:
99                 self.onDblClick()
100             if resultEvent == 3:
101                 self.onLongClick()
102             #----- Τέλος ορισμού κλάσης -----
103
104             #----- Συναρτήσεις εξυπηρέτησης συμβάντων -----
105             def key1_click():
106                 print("key1 Click")
107
108             def key2_click():
109                 print("key2 Click")
110
111             def key1_dblclick():
112                 print("key1 Double Click")
113
114             def key2_dblclick():
115                 print("key2 Double Click")
116
117             def key1_longclick():
118                 print("key1 Long Click")
119
120             def key2_longclick():
121                 print("key2 Long Click")
122             #----- Τέλος συναρτήσεων εξυπηρέτησης συμβάντων -----
123
124             #===== Κυρίως πρόγραμμα =====
125             #----- Δημιουργία σιγμιγνυτών και ορισμός συναρτήσεων εξυπηρέτησης -----
126             ts1 = Touch(t_pin1)
127             ts1.onClick = key1_click
128             ts1.onDblClick = key1_dblclick
129             ts1.onLongClick = key1_longclick
130
131             ts2 = Touch(t_pin2)
132             ts2.onClick = key2_click
133             ts2.onDblClick = key2_dblclick
134             ts2.onLongClick = key2_longclick
135
136             print("\nESP32 Touch Demo")
137
138             #----- Για πάντα -----
139             while True:
140                 ts1.checkButton() #Έλεγχος 1ου κουμπιού
141                 ts2.checkButton() #Έλεγχος 2ου κουμπιού
142                 time.sleep_ms(10) #Περίμενε 5 - 10msec

```

Ασκηση 5

Να γραφεί πρόγραμμα σε micropython το οποίο χρησιμοποιεί την ενσωματωμένη βιβλιοθήκη `asyncio` και αναβοσβήνει τα τρία από τα πέντε RGB Led, με σταθερά χρώματα το καθένα (κόκκινο, πράσινο, μπλε), με διαφορετική συχνότητα αναλαμπών. Συγκεκριμένα το πράσινο αναβοσβήνει με περίοδο 4 sec, το κόκκινο με περίοδο 1 sec και το μπλε με περίοδο 0,2 sec. Η `asyncio` επιτρέπει την ασύγχρονη εκτέλεση των προγραμμάτων χωρίς την χρήση εντολών blocking όπως η `time.sleep()`.

```

1 import machine, neopixel, asyncio
2
3 # Προετοιμασία RGB Led
4 np = neopixel.NeoPixel(machine.Pin(25), 5) #IO25, 5 x RGB Leds
5 led_status = [0, 0, 0, 0, 0] #Λίστα με την κατάσταση των Led (0=σβηστό, 1=αναμμένο)
6 RED = (255, 0, 0) #Κόκκινο
7 GREEN = (0, 255, 0) #Πράσινο
8 BLUE = (0, 0, 255) #Μπλε
9
10 # Η συνάρτηση αναβοσβήνει το Led με αριθμό num (0-4) με το χρώμα color
11 def toggle_led(num, color):
12     if led_status[num] == 0: #Είναι σβηστό
13         np[num] = color #Γράψε χρώμα
14         led_status[num] = 1 #Σημείωσε ότι τώρα είναι αναμμένο
15     else: #Διαφορετικά είναι αναμμένο
16         np[num] = (0, 0, 0) #Σβήσε (0,0,0) = σβηστό (μαύρο)
17         led_status[num] = 0 #Σημείωσε ότι τώρα είναι σβηστό
18     np.write() #Στείλε εντολή στα LED
19
20 # Ασύγχρονη λειτουργία (coroutine) για Πράσινο
21 async def blink_green_led():
22     while True: #Για πάντα
23         toggle_led(4, GREEN) #Αναβόσβησε το 1ο στην σειρά με χρώμα πράσινο
24         await asyncio.sleep(2) #Περίμενε 2 sec και δώσε την δυνατότητα εκτέλεσης άλλων λειτουργιών
25
26 # Ασύγχρονη λειτουργία (coroutine) για Κόκκινο
27 async def blink_red_led():
28     while True:
29         toggle_led(0, RED) #Αναβόσβησε το 5ο στην σειρά με χρώμα κόκκινο
30         await asyncio.sleep(0.5) #Περίμενε .5 sec και δώσε την δυνατότητα εκτέλεσης άλλων λειτουργιών
31
32 # Ασύγχρονη λειτουργία (coroutine) για Μπλε
33 async def blink_blue_led():
34     while True: #Για πάντα
35         toggle_led(2, BLUE) #Αναβόσβησε το μεσαίο με χρώμα μπλε
36         await asyncio.sleep(0.1) #Περίμενε 100 msec και δώσε την δυνατότητα εκτέλεσης άλλων λειτουργιών
37
38 # Ορισμός της συνάρτησης main που περιέχει το event loop
39 async def main():
40     # Δημιουργία εργασιών (tasks) ώστε να αναβοσβήνουν τα 3 Led ταυτόχρονα
41     asyncio.create_task(blink_green_led())
42     asyncio.create_task(blink_red_led())
43     asyncio.create_task(blink_blue_led())
44
45 # Δημιουργία και εκτέλεση του event loop
46 loop = asyncio.get_event_loop()
47 loop.create_task(main()) # Δημιουργία εργασίας (task) που εκτελεί το event loop
48 loop.run_forever() # Τρέξε για πάντα

```

Ασκηση 6

Να γραφεί πρόγραμμα σε micropython το οποίο διαβάζει την θέση του ποτενσιομέτρου του αρθρώματος επέκτασης AJS06.

Συνδέουμε το ποτενσιόμετρο AJS06 σε οποιαδήποτε από τις τέσσερις πάνω θύρες της πλακέτας (IO4, IO26, IO32, IO34) με το τροποποιημένο καλώδιο ή δύο EXP-AJ11 όπου το ένα έχει τροποποιηθεί και την breadboard. Ο μικροελεγκτής ESP32 διαθέτει μετατροπέα από αναλογικό σε ψηφιακό (Analog to Digital Converter **ADC**) εύρους 12bit δηλαδή το εύρος τιμών είναι από 0-4095. Μπορούμε να μειώσουμε την ευκρίνεια σε 11, 10 ή 9 bit. Επίσης διαθέτει στην είσοδο μεταβλητό εξασθενητή με τον οποίο αλλάζουμε το εύρος τάσεων που μπορεί να μετατρέψει. Δηλαδή αν επιλέξουμε την σταθερά `ADC.ATTN_6DB` τότε για τάση εισόδου 2 V ο ADC θα επιστρέψει τιμή 4095 στα 12bit.

ADC.ATTN_0DB - εύρος τιμών έως 1.2V

ADC.ATTN_2_5DB - εύρος τιμών έως 1.5V

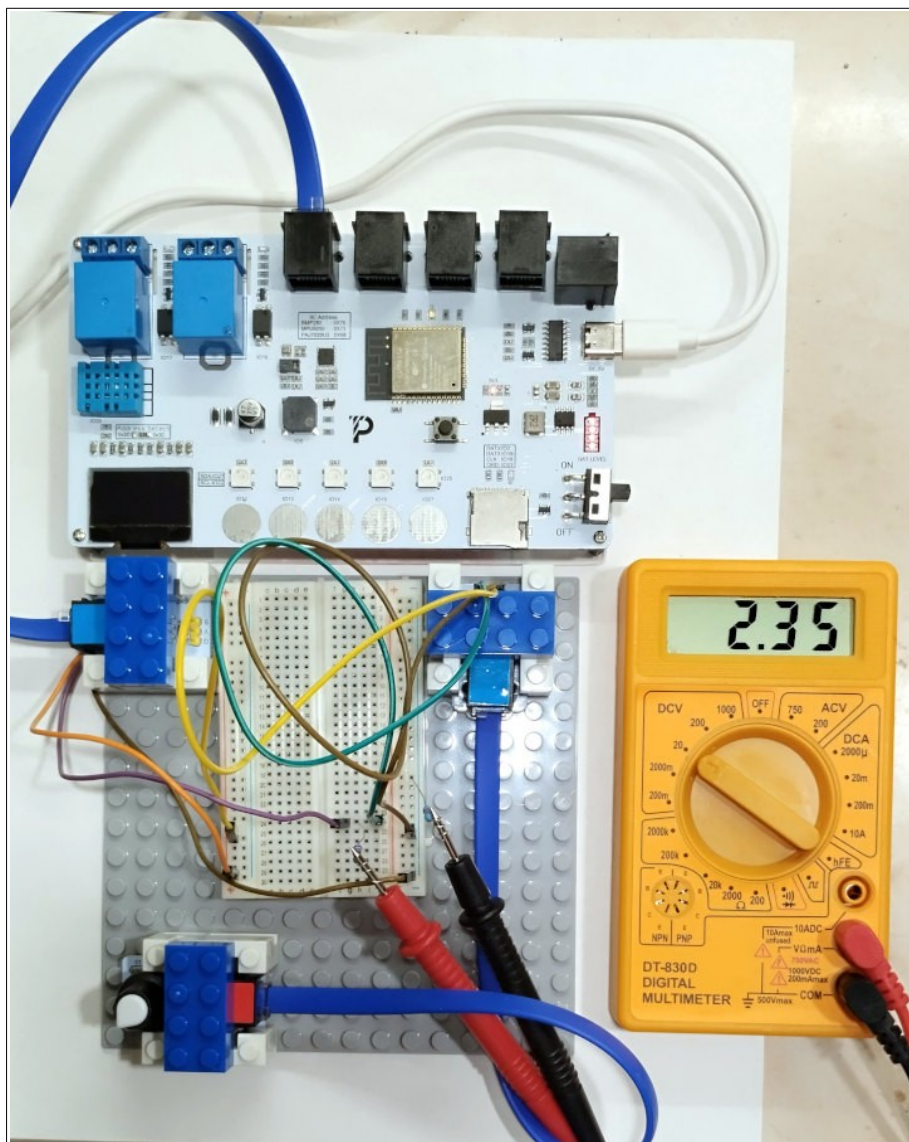
ADC.ATTN_6DB - εύρος τιμών έως 2.0V

ADC.ATTN_11DB - εύρος τιμών έως 3.3V

Επειδή το ένα άκρο του ποτενσιομέτρου συνδέεται στα 3,3V και το άλλο στην γείωση (GND), επιλέγουμε την σταθερά `ADC.ATTN_11DB`. Ο δρομέας οδηγείται στην είσοδο του ADC.

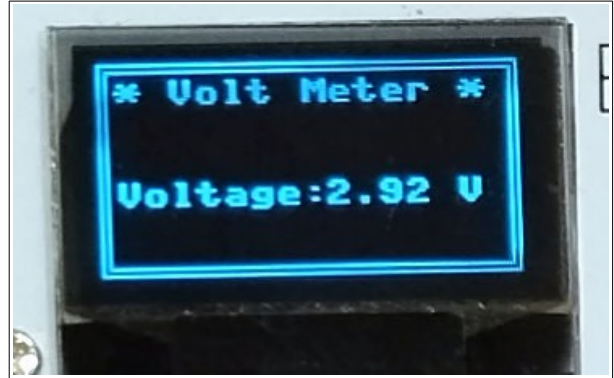
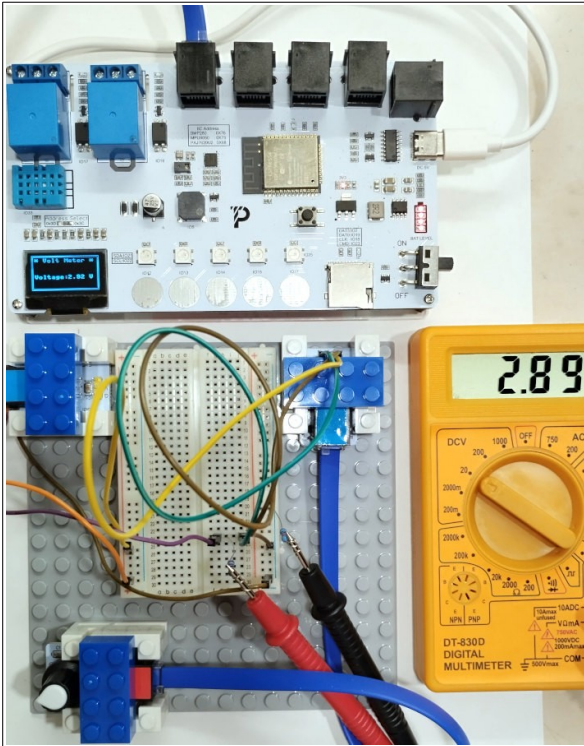
```
1 from machine import Pin, ADC
2 from time import sleep
3
4 pot = ADC(Pin(4)) #4, 26, 32, 34
5 pot.width(ADC.WIDTH_12BIT) #Ευκρίνεια 12BIT προκαθορισμένο, 9BIT, 10BIT, 11BIT
6 pot.atten(ADC.ATTN_11DB) # πλήρης κλίμακα έως 3.3V
7
8 while True:
9     pot_value = pot.read()
10    voltage = pot_value * (3.3 / 4096) #12bit
11    print("τιμή:", pot_value, "τάση:", round(voltage, 2), "V")
12    sleep(0.5)
```

Στην breadboard μπορούμε να συνδέσουμε και το πολύμετρο σε λειτουργία βολτομέτρου (μαύρος ακροδέκτης στο '-' και κόκκινος στον δρομέα ο οποίος συνδέεται με την είσοδο ADC. Για ευκολία μπορούμε να τυλίξουμε στον κάθε ακροδέκτη έναν αντιστάτη 220Ω και το άλλο άκρο να καρφωθεί στην breadboard. Εκτελούμε τον κώδικα και περιστρέφοντας αργά το ποτενσιόμετρο παρατηρούμε αν οι τιμές στο βολτόμετρο συμφωνούν με αυτές στο τερματικό του vscode. Φυσιολογικά πρέπει να υπάρχουν αποκλίσεις της τάξεως των 10 – 20mV οι οποίες οφείλονται σε σφάλμα του πολυμέτρου αλλά κυρίως στη μη γραμμική συμπεριφορά του ADC. Αυτό μπορεί να διορθωθεί με πίνακες διόρθωσης στον κώδικα και βαθμονόμηση (καλιμπράρισμα) με κάποιο πιο αξιόπιστο βολτόμετρο.



Άσκηση 7

Να τροποποιηθεί το παραπάνω ώστε να εμφανίζεται στην ενσωματωμένη μονόχρωμη οθόνη OLED η τιμή της τάσης σε Volts.



Στον κώδικα παρατηρούμε ότι έχει προστεθεί η μεταβλητή **avg_voltage** στην γραμμή 7. Αυτή είναι η μέση τιμή των μετρήσεων του ADC. Με την χρήση μέσης τιμής αποφεύγουμε την συνεχόμενη μεταβολή σε κάθε μέτρηση. Δηλαδή έχουμε μια εξομάλυνση στις ενδείξεις σαν να είχαμε συνδέσει παράλληλα στην είσοδο του ADC έναν πυκνωτή. Επίσης προσθέσαμε μια σταθερά **OFFS_COEF** που σαν σκοπό έχει να βελτιώσει την απόκλιση των

μετρήσεων σε σχέση με το φυσικό ψηφιακό βολτόμετρο. Η βελτίωση είναι μερική γιατί με τον συντελεστή (συνάρτηση 1ου βαθμού) δεν διορθώνονται μη γραμμικές συμπεριφορές στα άκρα. Στις παραπάνω εικόνες βλέπουμε την τιμή που εμφανίζει η οθόνη μας σε σχέση με αυτή του βολτομέτρου.

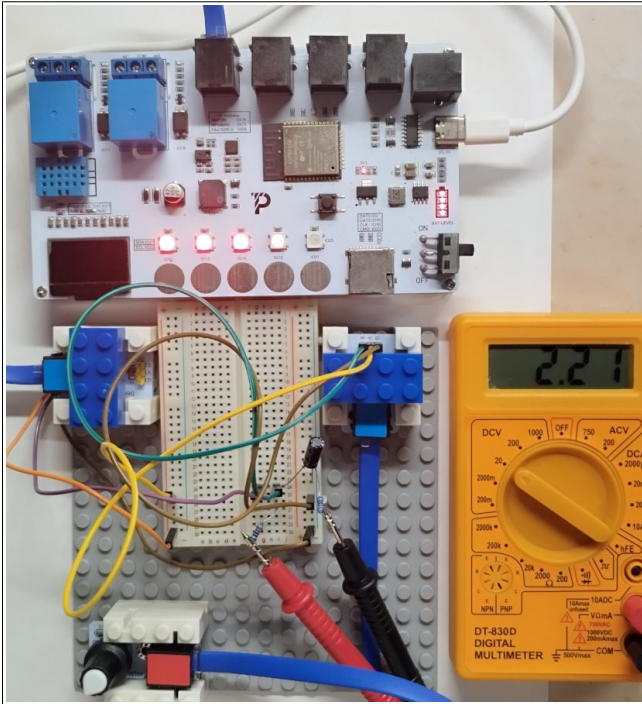
```

1 from machine import Pin, ADC, I2C
2 from time import sleep
3 from ssd1306 import SSD1306_I2C
4
5 AVG_FACTOR = 2.0 #Συντελεστής μέσης τιμής όσο μεγαλύτερος τόσο πιο αργή μεταβολή
6 OFFS_COEF = .95 #Συντελεστής διόρθωσης
7 avg_voltage = 0
8
9 pot = ADC(Pin(4)) #4, 26, 32, 34
10 pot.width(ADC.WIDTH_12BIT) #Ευκρίνεια 12BIT προκαθορισμένο, 9BIT, 10BIT, 11BIT
11 pot.atten(ADC.ATTN_11DB) # πλήρης κλίμακα έως 3.3V
12
13 i2c = I2C(0, sda = Pin(21), scl = Pin(22), freq = 400000)
14 display = SSD1306_I2C( width=128, height=64, i2c=i2c, addr=0x3c, external_vcc=False )
15 display.fill(0) #Καθαρίζει οθόνη
16 #Δημιουργεί ετικέτες
17 display.text('* Volt Meter *', 5, 5, 1) #θέση x, y, color 1/0
18 display.text("Voltage: ", 5, 35, 1)
19 display.text("V", 108, 35, 1)
20 #Δημιουργεί διπλό περίγραμμα
21 display.rect(0,0,127,63,1,0) #x, y, width, height, color=0/1, fill=0/1
22 display.rect(2,2,123,59,1,0) #x, y, width, height, color=0/1, fill=0/1
23 display.show() #Παρουσίαση στην οθόνη
24
25 while True:
26     pot_value = pot.read()
27     voltage = pot_value * (3.3 / 4096) / OFFS_COEF #12bit Στιγματική τιμή
28     avg_voltage = (AVG_FACTOR * avg_voltage + voltage) / (AVG_FACTOR + 1) #Μέση τιμή
29     print("τιμή:", pot_value, "τάση:", round(voltage, 2), "V")
30     display.rect(68, 34, 40, 9, 0, True) #Σβήσε παλιά τιμή
31     display.text(str(round(avg_voltage, 2)), 68, 35, 1) #Εμφάνισε νέα τιμή
32     display.show() #Παρουσίαση στην οθόνη
33     sleep(0.5)

```


Άσκηση 8

Να τροποποιηθεί η άσκηση 6 ώστε με την περιστροφή του ποτενσιομέτρου να ανάβουν διαδοχικά τα 5 RGB Leds της πλακέτας σε κόκκινο χρώμα. Το τελευταίο αναμμένο LED θα ανάψει αναλογικά δηλαδή αν έχουν ανάψει πλήρως δύο LED και η τιμή του ADC δεν επαρκεί ώστε να ανάψει πλήρως και τρίτο LED τότε αυτό θα ανάψει με περιορισμένη ένταση π.χ. 30%.



Παρατηρήστε ότι το τέταρτο LED ανάβει με χαμηλότερη ένταση από τα τρία αριστερά του. Η ευκρίνεια του ADC έχει μειωθεί στα 10 bits.

Για να μειωθεί το 'παίξιμο' των τιμών κατά την δειγματοληψία του ADC βάλαμε έναν ηλεκτρολυτικό πυκνωτή 100μF από τον δρομέα στην γείωση (GND). Ο δρομέας συνδέεται στην είσοδο του ADC (IO4). Το μακρύ ποδαράκι του πυκνωτή είναι το (+) και συνδέεται στον δρομέα, ενώ το κοντό είναι το (-) και συνδέεται στην γείωση (- ή GND).

```

1 from machine import Pin, ADC
2 from time import sleep
3 from neopixel import NeoPixel
4
5 RED = (255, 0, 0); GREEN = (0, 255, 0); BLUE = (0, 0, 255); BLANK = (0, 0, 0)
6 AVG_FACTOR = 3 # Συντελεστής μέσης τιμής όσο μεγαλύτερος τόσο πιο αργή μεταβολή
7 avg_val = 0
8
9 pot = ADC(Pin(4)) #4, 26, 32, 34
10 pot.width(ADC.WIDTH_10BIT) # Ευκρίνεια 10BIT, 9BIT, 10BIT, 11BIT, 12BIT προκαθορισμένο
11 pot.atten(ADC.ATTN_11DB) # πλήρης κλίμακα έως 3.3V
12
13 np = NeoPixel(Pin(25), 5) #IO25, 5 x RGB Leds
14
15 # Συνάρτηση παρόμοια με την map του Arduino
16 # Δέχεται τρέχουσα τιμή, ελάχιστη τιμή, μέγιστη τιμή, έξοδος από, έως
17 def map(value, in_min, in_max, out_min, out_max):
18     scaled_value = (value - in_min) * (out_max - out_min) / (in_max - in_min) + out_min
19     return int(scaled_value) # Επιστρέφει ακέραιο
20
21 # Συνάρτηση για να ανάβει τα Leds ανάλογα με την τιμή του ADC
22 def out_leds(x):
23     n = x // 205 # Γιατί 1024 / 5 leds = 204,8 ακέραια διαίρεση
24     y = x % 205 # Υπόλοιπο για το τελευταίο Led
25     i = 0
26     while i < n: # Ανάβει πλήρως όσα Led υπολογίστηκαν στο n
27         np[4 - i] = RED # Ανάβουν ανάποδα
28         i += 1
29     a = i # Κράτησε την τρέχουσα τιμή του i
30     while i < 5: # Αν δεν είναι να ανάψουν όλα τότε σβήσε τα υπόλοιπα
31         np[4 - i] = BLANK # Σβήνουν ανάποδα
32         i += 1
33     print(n, y, map(y, 0, 204, 2, 255))
34     i = a # Επαναφορά του i
35     if i <= 4: # Αν είναι από 0 έως 4 τότε άναψε μερικώς αυτό το Led
36         np[4 - i] = (map(y, 0, 204, 2, 250), 0, 0) # Αλλαγή κλίμακας από 0-204 σε 2-255 για το κόκκινο (255 πλήρης ένταση)
37     np.write()
38
39 # Για πάντα
40 while True:
41     pot_value = pot.read() # Διάβασε τιμή ποτενσιομέτρου
42     avg_val = int((AVG_FACTOR * avg_val + pot_value) / (AVG_FACTOR + 1)) # Μέση τιμή
43     out_leds(avg_val) # Άναψε τα Leds
44     print("τιμή:", avg_val)
45     sleep(.1)

```

Ασκηση 9

Να τροποποιηθεί ο παραπάνω κώδικας ώστε να χρησιμοποιηθεί το ποτενσιόμετρο ως συσκευή εισόδου εντολών. Συγκεκριμένα ας υποθέσουμε ότι θέλουμε να επιλέγουμε πέντε προγράμματα πλυντηρίου περιστρέφοντας το ποτενσιόμετρο και ανάλογα την θέση να εμφανίζεται το πρόγραμμα στο τερματικό π.χ. 'Θέση-2' και να ανάβει ένα από τα 5 RGB Leds με διαφορετικό χρώμα. Για τον συγκεκριμένο σκοπό υπάρχουν άλλα εξαρτήματα, όπως περιστροφικοί διακόπτες 16 θέσεων που βγάζουν 4bit ανάλογα την θέση ή R.I.E. (Rotary Impulse Encoders) που καταλαβαίνουν πόσα βήματα έχουν περιστραφεί και προς ποια κατεύθυνση. Επειδή το kit δεν διαθέτει κάτι αντίστοιχο εμείς θα χρησιμοποιήσουμε το ποτενσιόμετρο.

```

1 from machine import Pin, ADC
2 from time import sleep
3 from neopixel import NeoPixel
4
5 #Σταθερές χρωμάτων
6 RED = (255, 0, 0); GREEN = (0, 255, 0); BLUE = (0, 0, 255); YELLOW = (255, 255, 0)
7 MAGENTA = (255, 0, 255); CYAN = (0, 255, 255); BLANK = (0, 0, 0)
8 AVG_FACTOR = 1.5 #Συντελεστής μέσης τιμής όσο μεγαλύτερος τόσο πιο αργή μεταβολή
9 avg_val = 0 #Μέση τιμή του ADC
10
11 pot = ADC(Pin(4)) #4, 26, 32, 34
12 pot.width(ADC.WIDTH_10BIT) #Ευκρίνεια 10BIT, 9BIT, 10BIT, 11BIT, 12BIT προκαθορισμένο
13 pot.atten(ADC.ATTN_11DB) # πλήρης κλίμακα έως 3.3V
14
15 np = NeoPixel(Pin(25), 5) #IO25, 5 x RGB Leds
16
17 #Λίστα με τις εντολές
18 C = ['Θέση - 1', 'Θέση - 2', 'Θέση - 3', 'Θέση - 4', 'Θέση - 5']
19 #Παράλληλη λίστα με τα χρώματα για κάθε θέση
20 COLORS = [RED, GREEN, BLUE, YELLOW, MAGENTA]
21 PLACES = len(C) #Υπολόγισε αριθμό θέσεων
22 THRESS = 5 #Κατώφλι υστέρησης
23 pos = 0 #Θέση στην λίστα με τις εντολές
24
25 #Η συνάρτηση σβήνει και τα 5 RGB Leds
26 def blank_leds():
27     for i in range(5):
28         np[i] = BLANK
29
30 #Συνάρτηση για να ανάβει τα Leds ανάλογα με την τιμή του ADC
31 def out_leds(n):
32     blank_leds()
33     np[4 - n] = COLORS[n]
34     np.write()
35
36 #Η συνάρτηση επιλέγει εντολή ανάλογα με την τιμή του x δηλαδή την θέση του ποτενσιόμετρου
37 #Υπάρχει πρόβλεψη υστέρησης τιμής THRESS ώστε να μην παίζει η απόφαση της εντολής σε γειτονικές
38 #τιμές. π.χ. αν είναι 127 και 128 τότε θα μεταβάλλεται από θέση-1 σε θέση-2. Με υστέρηση π.χ. 5
39 #για να αλλάξει σε θέση-1 πρέπει να πέσει κάτω από 123 και για να ανέβει σε θέση-2 πρέπει να
40 #ξεπεράσει το 133.
41 def get_command(x):
42     global pos
43     stp = 1024 // PLACES #Βήμα για κάθε θέση
44     s = 0 #Αθροισμα βημάτων
45     i = 0
46     for i in range(8):
47         if x > s + THRESS and x < s + stp - THRESS: #Αν ξέφυγε από την περιοχή υστέρησης τότε
48             pos = i #Αλλάξε την τιμή του pos, διαφορετικά ισχύει η προηγούμενη τιμή
49             s += stp
50     return pos
51
52 #Για πάντα
53 while True:
54     pot_value = pot.read() #Διάβασε τιμή ποτενσιόμετρου
55     avg_val = int((AVG_FACTOR * avg_val + pot_value) / (AVG_FACTOR + 1)) #Μέση τιμή
56     tmp = get_command(avg_val)
57     print(C[tmp]) #Εμφάνισε εντολή
58     out_leds(tmp)
59     #print("τιμή:", avg_val) #Debug
60     sleep(1)

```

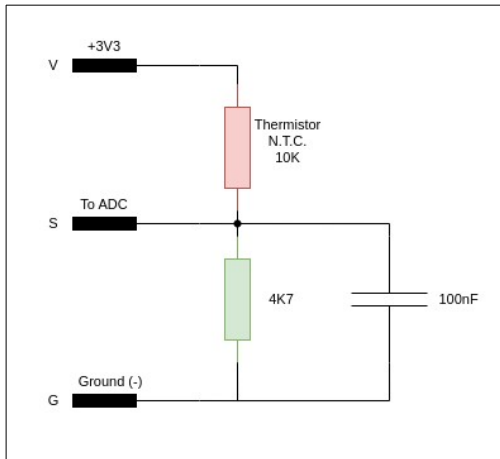
Ασκηση 10

Μέτρηση θερμοκρασίας με αναλογικό αισθητήρα. Στα προηγούμενα χρησιμοποιήσαμε για μέτρηση θερμοκρασίας, ψηφιακούς αισθητήρες όπως ο DHT11 ή ο BMP280. Αυτοί έχουν βαθμονομηθεί από τις κατασκευάστριες εταιρείες και στέλνουν τις μετρήσεις ψηφιακά μέσω διαύλων όπως IIC, onewire, SPI κλπ. και ο προγραμματιστής μπορεί να εμπιστευτεί την μέτρηση ως αξιόπιστη και να την χρησιμοποιήσει στην εφαρμογή του.

Μπορούμε αν θέλουμε να χρησιμοποιήσουμε τον αισθητήρα **AJS01** ο οποίος διαθέτει έναν ειδικό αντιστάτη το **thermistor** στον οποίο μεταβάλλεται κατά πολύ η αντίσταση με τις μεταβολές της θερμοκρασίας. Σε θερμοκρασία 25°C η αντίσταση για τον συγκεκριμένο είναι 10K. Όταν η

θερμοκρασία ανέβει η αντίσταση θα μειωθεί και όταν κατέβει θα αυξηθεί. Δηλαδή έχει αρνητικό συντελεστή θερμοκρασίας (**N.T.C.** Negative Temperature Coefficient).

Το άρθρωμα **AJS01** έχει την ακόλουθη απλή συνδεσμολογία. Δηλαδή έχουμε έναν διαιρέτη τάσης με το thermistor NTC και έναν αντιστάτη γνωστής αντίστασης. Το κοινό σημείο πηγαίνει στον



μετατροπέα ADC και μετρώντας την τάση υπολογίζουμε την αντίσταση του NTC.

Στη συνέχεια εφαρμόζοντας κάποιους τύπους με τους τρεις συντελεστές που βρίσκουμε στο φύλλο δεδομένων του εξαρτήματος μπορούμε να υπολογίσουμε την θερμοκρασία. Η χρήση thermistor είναι οικονομική και απλή αλλά δεν έχουμε μεγάλη ακρίβεια και χρειάζεται βαθμονόμηση με κάποιο θερμόμετρο αναφοράς. Συνήθως χρησιμοποιείται σε εφαρμογές στις οποίες δεν μας ενδιαφέρει τόσο πολύ η ακρίβεια ($\pm 2^\circ\text{C}$) όπως θερμοστάτες συσκευών κλπ. Επιπλέον είναι ανθεκτικός σε πολύ υψηλές θερμοκρασίες $>100^\circ\text{C}$ όπου οι ψηφιακοί αισθητήρες παρουσιάζουν προβλήματα.

Συνδέουμε το άρθρωμα AJS01 με τροποποιημένο καλώδιο στην θύρα IO4 και δοκιμάζουμε τον ακόλουθο κώδικα. Για σύγκριση χρησιμοποιούμε και τον ενσωματωμένο αισθητήρα DHT11.

```

1 from machine import Pin, ADC
2 from dht import DHT11
3 from time import sleep
4 import math
5
6 R2 = 4660 #Ω
7 Vi = 3.27 #Volts
8
9 #Σταθερές Steinhart για το NTC Thermistor TCS610
10 A = 1.1279e-03 #0.001129148
11 B = 2.3429e-04 #0.000234125
12 C = 8.7298E-08 #0.000000876741
13
14 ntc = ADC(Pin(4)) #4, 26, 32, 34
15 ntc.width(ADC.WIDTH_12BIT) #Ευκρίνεια 10BIT, 9BIT, 10BIT, 11BIT, 12BIT προκαθορισμένο
16 ntc.atten(ADC.ATTN_11DB) # πλήρης κλίμακα έως 3.3V
17
18 sensor = DHT11(Pin(33)) #DHT11 για σύγκριση
19 sensor.measure() #Νέα μέτρηση του DHT11
20
21 #Για πάντα
22 while True:
23     vo = ntc.read() #Διάβασε τιμή της τάσης του διαιρέτη
24     vo /= .9 #Διόρθωση μη γραμμικής συμπεριφοράς ADC
25     Rntc = R2 * (4095.0 / vo - 1); #Υπολογισμός αντίστασης NTC
26     #Υπολογισμός θερμοκρασίας σε βαθμούς Kelvin
27     TempK = 1 / (A + (B * math.log(Rntc)) + C * math.pow(math.log(Rntc), 3))
28     TempC = TempK - 273.15 #Μετατροπή σε °C
29     print("Θερμοκρασία NTC: ", round(TempC, 2), "°C") #Εμφάνισε θερμοκρασία NTC
30     print("Θερμοκρασία DHT11: ", sensor.temperature() + 1.0, "°C\n")
31     sleep(2) #Περίμενε 2 sec

```

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

>>>
>>>
MPY: soft reboot
θερμοκρασία NTC: 26.57 °C
θερμοκρασία DHT11: 26.0 °C

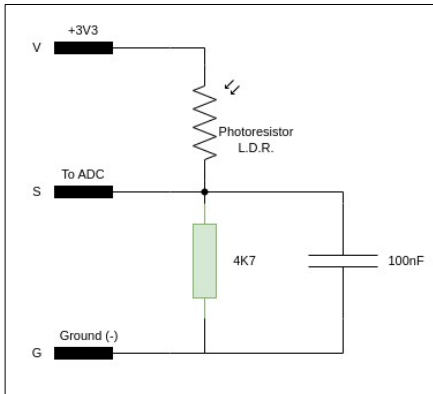
θερμοκρασία NTC: 26.52 °C
θερμοκρασία DHT11: 26.0 °C

```

Άσκηση 11

Αισθητήρας φωτός – Φωτοαντιστάτης LDR. Πολλές φορές χρειάζεται να φτιάξουμε αυτοματισμούς οι οποίοι με δεδομένο την ένταση του ορατού φωτός, κάνουν κάποιες ενέργειες π.χ. ανάβουν ή σβήνουν τα φώτα.

Να γίνει κύκλωμα και πρόγραμμα σε Micropython με τα αρθρώματα DJX01 (λευκό LED) και AJS03 (φωτοαντιστάτης LDR), ώστε αν η ένταση του φωτός μειωθεί να ανάβει το LED και όταν είναι αρκετή να το σβήνει.



Δίπλα φαίνεται το κύκλωμα του αρθρώματος AJS03. Όπως καταλαβαίνουμε πρόκειται για αναλογική μεταβολή της τάσης, επομένως θα συνδεθεί σε είσοδο ADC.

Συνδέουμε με τροποποιημένα καλώδια RG12 το LDR στην υποδοχή IO4 και το LED στην IO32.

Στη συνέχεια δοκιμάζουμε το παρακάτω πρόγραμμα. Τροποποιείτε τις τιμές στις γραμμές 17 και 19 ώστε το πρόγραμμα να λειτουργεί σωστά στο περιβάλλον του εργαστηρίου.

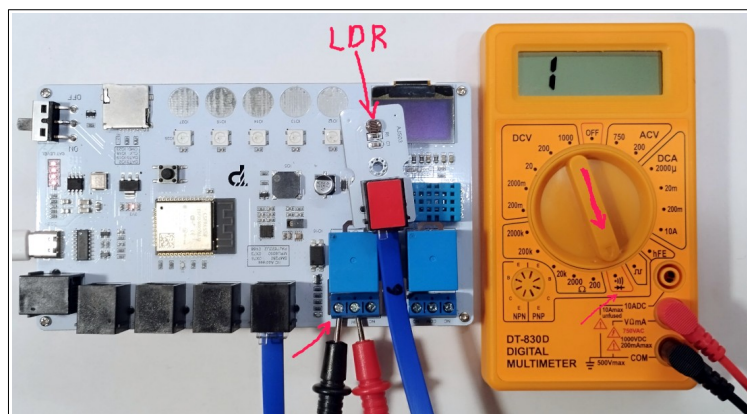
```

1 from machine import Pin, ADC
2 from time import sleep
3
4 #Αντίσταση LDR από 600K στο σκοτάδι έως 200Ω σε δυνατό φως
5 ldr = ADC(Pin(4)) #4, 26, 32, 34
6 ldr.width(ADC.WIDTH_12BIT) #Ευκρίνεια 10BIT, 9BIT, 10BIT, 11BIT, 12BIT προκαθορισμένο
7 ldr.atten(ADC.ATTN_11DB) # πλήρης κλίμακα έως 3.3V
8 #LED DJX01 στο pin IO32
9 light = Pin(32, Pin.OUT) #Ψηφιακή έξοδος
10 light.value(0) #Σβηστό
11
12 #Για πάντα
13 while True:
14     vo = ldr.read() #Διάβασε τιμή της τάσης του διαιρέτη
15     #Οι τιμές είναι από 0 (απόλυτο σκοτάδι) έως 4095 (δυνατό φως)
16     print("Τιμή LDR: ", vo) #Εμφάνισε τιμή ADC
17     if vo < 500: #Δυνατότητα υστέρησης για αποφυγή αναλαμπών
18         light.value(1) #Αναψε το LED
19     elif vo > 800:
20         light.value(0) #Σβήσε το LED
21     sleep(2) #Περίμενε 2 sec

```

Να τροποποιηθεί το παραπάνω ώστε να ανάβει τα φώτα σε δίκτυο 220V μέσω του ηλεκτρονόμου IO16.

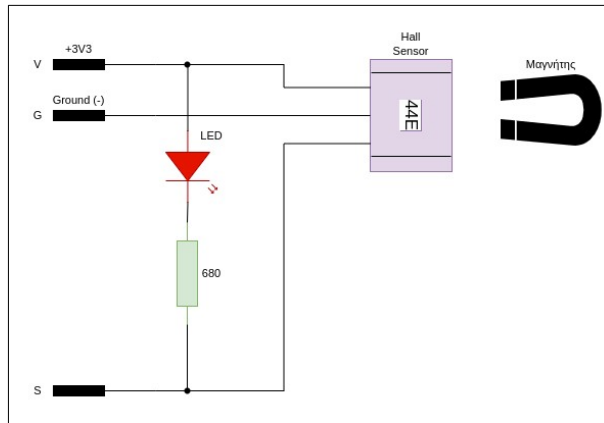
Εδώ η μοναδική αλλαγή γίνεται στην γραμμή 9 όπου το 32 θα γίνει 16. Γυρίστε τον μεταγωγό του πολυμέτρου στην θέση με το **beep** – **δίοδο** και συνδέστε τους ακροδέκτες στην κλέμα στα σημεία COM και ON. Ελέγξτε αν στο σκοτάδι κλείνει ο ηλεκτρονόμος και ακούγεται σφύριγμα από το πολύμετρο.



Άσκηση 12

Αισθητήρας Hall. Ο αισθητήρας Hall είναι ένα εξάρτημα ανίχνευσης μαγνητικού πεδίου. Όταν ένας μαγνήτης πλησιάσει στον αισθητήρα αυτός αλλάζει την κατάσταση του. Το συγκεκριμένο άρθρωμα του kit βγάζει στον ακροδέκτη S λογικό '1' όταν δεν υπάρχει μαγνητικό πεδίο και λογικό '0' όταν πλησιάσει κοντά ένας μικρός μαγνήτης. Χρησιμοποιείται ευρέως σε ηλεκτρικές συσκευές, αυτοκίνητα κ.λ.π.

Να γίνει κύκλωμα και πρόγραμμα σε Micropython με τα αρθρώματα DJX01 (λευκό LED) και DJS07 (αισθητήρα Hall), ώστε όταν ο μαγνήτης πλησιάζει να ανάβει το LED και όταν απομακρύνεται να σβήνει.



```
1 from machine import Pin
2 from time import sleep
3
4 #Αισθητήρας Hall DJS07 στο pin IO4
5 hall = Pin(4, Pin.IN, Pin.PULL_UP)
6 #LED DJX01 στο pin IO32
7 light = Pin(32, Pin.OUT) #Ψηφιακή έξοδος
8 light.value(0) #Σβηστό
9
10 #Για πάντα
11 while True:
12     state = hall.value()
13     print("Κατάσταση Hall: ", state) #Εμφάνισε κατάσταση αισθητήρα Hall
14     if not state: #Αν είναι 0 έχει πλησιάσει ο μαγνήτης
15         light.value(1) #Ανάψε το LED
16     else: #Διαφορετικά δεν υπάρχει ικανό μαγνητικό πεδίο
17         light.value(0) #Σβήσε το LED
18         sleep(1) #Περίμενε 1 sec
```

Άσκηση 13

Επιλέγον γραμματοσειρές στην οθόνη OLED.

Αρχικά κατεβάζουμε από το [github](https://www.fontspace.com/category/ttf) το πρόγραμμα **font_to_py.py**. Έπειτα πάμε στον ιστότοπο <https://www.fontspace.com/category/ttf> και κατεβάζουμε ένα οποιοδήποτε font π.χ. Ariana Violeta. Αποσυμπιέζουμε το αρχείο και αν θέλουμε βλέπουμε το δείγμα με την εφαρμογή προβολής γραμματοσειρών του λειτουργικού μας συστήματος. Αντιγράφουμε το αρχείο .ttf στον ίδιο φάκελο με το πρόγραμμα font_to_py.py. Για συστήματα Linux φτιάχνουμε ένα **venv** π.χ. `python3 -m venv ~/micropython`. Χρησιμοποιούμε το περιβάλλον με `source ~/micropython/bin/activate`. Εγκαθιστούμε το argpass με `pip install argpass` και την βιβλιοθήκη freetype-py με `pip install freetype-py`. Για συστήματα Windows απλώς `pip install argpass` χωρίς την χρήση venv.

Στη συνέχεια μεταφράζουμε το αρχείο από ttf σε py με την εντολή:

```
python font_to_py.py ArianaVioleta.ttf 32 ArianaVioleta_python.py
```

Το 32 είναι μέγεθος και με δοκιμές επιλέγουμε αυτό που μας ταιριάζει.

Μέσα στον φάκελο εκτός από το boot.py και main.py πρέπει να συμπεριληφθούν τα αρχεία ssd1306.py, writer.py, η γραμματοσειρά ArianaVioleta_python.py και οποιαδήποτε άλλη γραμματοσειρά.

```
1 from machine import Pin, I2C
2 from ssd1306 import SSD1306_I2C
3 from writer import Writer
4 import ArianaVioleta_python #Μεταφρασμένη γραμματοσειρά
5
6 i2c = I2C(0, sda = Pin(21), scl = Pin(22), freq = 400000)
7 display = SSD1306_I2C( width=128, height=64, i2c=i2c, addr=0x3c, external_vcc=False )
8 display.fill(0) #Καθαρίζει οθόνη
9
10 wri = Writer(display, ArianaVioleta_python) #Χρήση γραμματοσειράς
11 wri.set_textpos(display, 0, 0) #Αρχική θέση y, x
12 wri.printstring("Hello\n") #Εμφάνιση κειμένου
13 wri.printstring("World")
14 display.text("Test", 90, 56) #Εμφάνιση κειμένου με τον κλασικό τρόπο
15 display.show() #Παρουσίασε στην οθόνη
```

Αν θέλουμε μπορούμε να μεταφράσουμε γραμματοσειρές από τον υπολογιστή μας. Για παράδειγμα στο Ubuntu Linux οι γραμματοσειρές βρίσκονται στο /usr/share/fonts/truetype.

Στον παρακάτω κώδικα χρησιμοποιούμε δύο γραμματοσειρές ταυτόχρονα.



```
1 from machine import Pin, I2C
2 from ssd1306 import SSD1306_I2C
3 from writer import Writer
4 import ArianaVioleta_python, FreeSerifBold_python #Μεταφρασμένες γραμματοσειρές
5
6 i2c = I2C(0, sda = Pin(21), scl = Pin(22), freq = 400000)
7 display = SSD1306_I2C( width=128, height=64, i2c=i2c, addr=0x3c, external_vcc=False )
8 display.fill(0) #Καθαρίζει οθόνη
9
10 wri1 = Writer(display, ArianaVioleta_python) #Χρήση γραμματοσειράς στο wri1
11 wri2 = Writer(display, FreeSerifBold_python) #Χρήση γραμματοσειράς στο wri2
12 wri1.set_textpos(display, 0, 0) #Αρχική θέση y, x
13 wri1.printstring("Hello") #Εμφάνιση κειμένου
14 wri2.set_textpos(display, 30, 0) #Αρχική θέση y, x
15 wri2.printstring("World")
16 display.text("Test", 90, 56) #Εμφάνιση κειμένου με τον κλασικό τρόπο στην θέση x, y
17 display.show() #Παρουσίασε στην οθόνη
```



Άσκηση 14

Ελληνικές γραμματοσειρές και σύμβολα στην οθόνη OLED.

Για υποστήριξη ελληνικών θα χρησιμοποιήσουμε μια άλλη βιβλιοθήκη η οποία βρίσκεται στην διεύθυνση <https://github.com/easytarget/microPyEZfonts> και συγκεκριμένα το αρχείο **ezFBfont.py** το οποίο αντιγράφεται στο σύστημα αρχείων του esp32.

Για μετατροπή των χαρακτήρων θα χρησιμοποιήσουμε το πρόγραμμα **bdf2dict.py** και για αρχείο εισόδου πρέπει να χρησιμοποιήσουμε αρχείο τύπου bdf. Το πρόγραμμα εκτελείται στο υπολογιστή. Από τον κατάλογο Unicode/unifont_15.1.05 κατεβάζουμε το αρχείο unifont_15.1.05.bdf στον ίδιο φάκελο με το πρόγραμμα bdf2dict.py και φτιάχνουμε ένα αρχείο με τους χαρακτήρες που θέλουμε στην γραμματοσειρά μας και το ονομάζουμε charset.txt π.χ. `!"#$%&'()*+,-./0123456789:;<=>?@ABCDEFGHIJKLMNPQRSTUVWXYZ[\]^_`abcdefghijklmnopqrstuvwxyz{|}~ΑΒΓΔΕΖΗΘΙΚΛΜΝΞΟΠΡΣΤΥΦΧΨΩαβγδεζηθικλμνξοπρστυφχψωΆΈΉΊΌΥΰάέήϊούός`, τέλος κάνουμε την μετατροπή γράφοντας:

```
python bdf2dict.py unifont-15.1.05.bdf my_charset.txt
```

Το αρχείο το οποίο θα παραχθεί το αντιγράφουμε στο filesystem του esp32. Αν δεν πρόκειται να χρησιμοποιήσουμε κάποιους χαρακτήρες τους αφαιρούμε από το charset.txt ώστε να καταναλώνει λιγότερη μνήμη.

```
1 from machine import Pin, I2C
2 from ssd1306 import SSD1306_I2C
3 from ezFBfont import ezFBfont
4
5 import my_unifont_15_1_05 as font1 #Εισαγωγή μεταφρασμένης γραμματοσειράς
6
7 i2c = I2C(0, sda = Pin(21), scl = Pin(22), freq = 400000)
8 display = SSD1306_I2C( width=128, height=64, i2c=i2c, addr=0x3c, external_vcc=False )
9 display.fill(0) #Καθαρίζει οθόνη
10
11 #Δημιουργία στιγμιότυπου
12 font = ezFBfont(display, font1,
13                 halign='center',
14                 valign='center',
15                 hgap=1,
16                 verbose=True)
17
18 font.write("Καλημέρα\nΚόσμε \nAbcdefg12345\n+*/#|^&.", 63, 31) #Εγγραφή στο Framebuffer
19 display.show() #Παρουσίασε στην οθόνη
```

Μπορούμε να μετατρέψουμε ttf ή otf fonts από τον υπολογιστή μας με το εργαλείο **otf2bdf** το οποίο υπάρχει στο linux ή με online ιστότοπους μετατροπής. Για μια γραμματοσειρά ttf γράφουμε:

```
otf2bdf -p 8 DejaVuSerif-Bold.ttf -o DejaVuSerif-Bold.bdf
```

Το **-p 8** είναι οι στιγμές δηλαδή το μέγεθος της γραμματοσειράς. Αν δεν το ορίσουμε τότε το καθορισμένο είναι **12pts**.

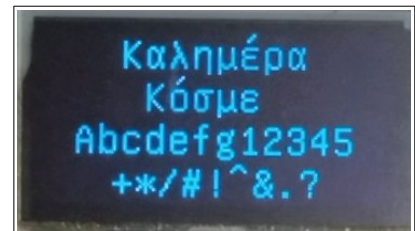
Μετά το μετατρέπουμε σε py με την ίδια εντολή όπως πριν:

```
python bdf2dict.py -p 8 DejaVuSerif-Bold.bdf my_charset.txt
```

Με το πρόγραμμα **FontForge** το οποίο είναι διαθέσιμο και για Windows, διορθώνουμε ατέλειες στο αρχείο bdf και αποθηκεύουμε με **‘Δημιουργία Γραμματοσειρών’**.

Δημιουργώντας διαφορετικά στιγμιότυπα της κλάσης ezFBfont μπορούμε να έχουμε πολλές γραμματοσειρές στην ίδια οθόνη.

Δοκιμάστε επίσης γραμματοσειρές από τον κατάλογο Unicode/efont-unicode-bdf-0.4.2 και τα σύμβολα όπως μπαταρίες, cursors, βελάκια, ψηφία 7 τμημάτων κλπ. από τον κατάλογο Symbols στο δέντρο του github.



Άσκηση 15

Εικόνες στην οθόνη OLED.

Για να εμφανίσουμε εικόνες στην οθόνη OLED αρχικά ζωγραφίζουμε μια εικόνα σε ένα πρόγραμμα ζωγραφικής π.χ. το Gimp. Οι διαστάσεις είναι 128x64 και φροντίζουμε να είναι ασπρόμαυρο (2 τόνοι). Το αποθηκεύουμε με επέκταση png ή jpg. Μπορούμε να βρούμε και μια έτοιμη εικόνα και να αλλάξουμε τις διαστάσεις σε 128x64.

Από το github πάμε στην διεύθυνση <https://github.com/TimHanewich/MicroPython-SSD1306> και κατεβάζουμε το αρχείο **src/convert.py**. Επίσης εγκαθιστούμε την βιβλιοθήκη **pillow** με **pip install pillow**. Βάζουμε τις εικόνες στον ίδιο φάκελο με το convert.py. Μέσα από τον φάκελο ανοίγουμε ένα shell της python3. Εκεί γράφουμε:

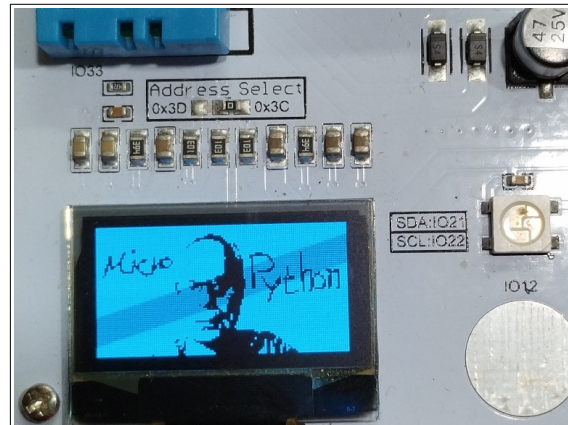
```
>>> import convert
>>> converted = convert.image_to_buffer(r"hello.png")
>>> buffer = converted[0]
>>> buffer
```

Εδώ βλέπουμε ένα μεγάλο byte array το οποίο ξεκινάει με `b'\x00\x00\x00\...` και τελειώνει με `!`. Αυτό το αντιγράφουμε ώστε να το μεταφέρουμε στο πρόγραμμά μας.

Στην 2η εντολή δίπλα στο όνομα του αρχείου μπορούμε να βάλουμε και 2η παράμετρο. Σ' αυτή ορίζουμε το κατώφλι με το οποίο αποφασίζει αν το pixel είναι άσπρο ή μαύρο.

[illegible]

Στο πρόγραμμα αυτό έχουμε δύο εικόνες οι οποίες εναλλάσσονται με περίοδο 1sec. Τα bytearray hello_buf και hello_buf1 μπορούμε να τα βάλουμε σε ένα άλλο αρχείο .py και να τα κάνουμε import στην αρχή του προγράμματος.



Άσκηση 16

Χρήση του αισθητήρα P.I.R. – Ανίχνευση κίνησης

Οι αισθητήρες παθητικού υπέρυθρου (PIR - Passive Infrared) είναι ηλεκτρονικές συσκευές που ανιχνεύουν κίνηση μετρώντας αλλαγές στην υπέρυθη ακτινοβολία που εκπέμπεται από αντικείμενα στο οπτικό τους πεδίο, συνήθως ανθρώπους ή ζώα. Είναι μικροί, χαμηλής κατανάλωσης και χρησιμοποιούνται ευρέως σε συστήματα ασφαλείας, αυτοματισμούς φωτισμού και αισθητήρες κίνησης. Εμείς θα υλοποιήσουμε ένα σύστημα συναγερμού με αυτόν τον αισθητήρα.

Τι θα χρειαστούμε

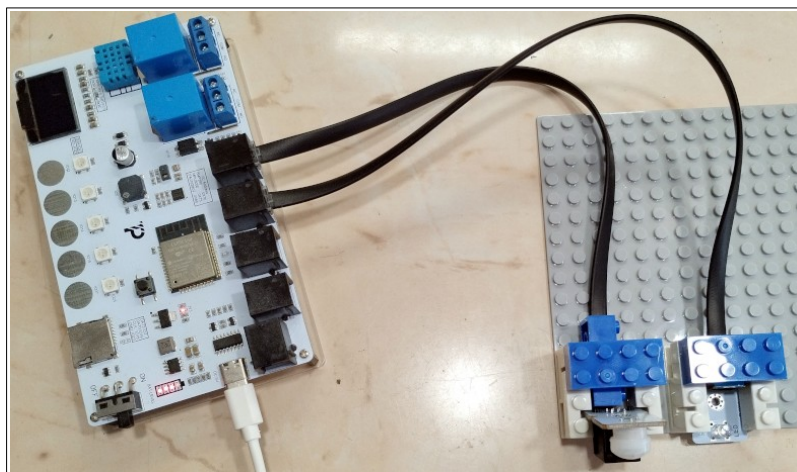
1. Μικροελεγκτή **ARD:icon II**.
2. Άρθρωμα αισθητήρα P.I.R. **DJS19** (Το κιτ περιέχει 2 τεμάχια).
3. Άρθρωμα λευκού LED **DJX01**.
4. Δύο καλώδια RJ12 για ESP32 (μαύρο χρώμα) ή τροποποιημένα.
5. Τουβλάκια και βάση στήριξης.
6. Καλώδιο USB για προγραμματισμό ESP32 και παροχή ρεύματος (Άσπρο type 'C').

Τοποθετούμε τα εξαρτήματα και τα συνδέουμε ως εξής:

Το PIR στο IO4 του ARD:iconII.

Το λευκό LED στο IO26.

Θα χρησιμοποιήσουμε τον ενσωματωμένο βομβητή της πλακέτας ο οποίος συνδέεται στο IO5.



Δεν απαιτούνται να γίνουν import επιπλέον βιβλιοθήκες πέραν των ενσωματωμένων. Γράφουμε τον κώδικα στο Thonny ή στο VS Code.

```

1 import time
2 from machine import Pin
3
4 PIR = Pin(4, Pin.IN, Pin.PULL_UP) #PULL_UP ώστε αν κοπεί το καλώδιο του PIR να χτυπάει πάντα
5 BEEPER = Pin(5, Pin.OUT) #Ο ενσωματωμένος βομβητής της πλακέτας συνδέεται στο pin IO5
6 LED = Pin(26, Pin.OUT) #Το LED συνδέεται στο pin IO26
7
8 # Η συνάρτηση παράγει ένα τόνο συχνότητας freq και διάρκειας dur
9 def tone(freq = 500, dur = .5): #Προκαθορισμένες τιμές
10     #Υπολογισμοί
11     period = 1.0 / freq #Περίοδος
12     half_per = int((period / 2) * 1000000) #Ημιπερίοδος
13     times = int(dur / period) #Αριθμός κύκλων
14     for i in range(times):
15         BEEPER.value(1) #Beeper ενεργό
16         time.sleep_us(half_per) #Περίμενε τον χρόνο της ημιπερίοδου
17         BEEPER.value(0) #Beeper ανενεργό
18         time.sleep_us(half_per) #Περίμενε τον χρόνο της ημιπερίοδου
19
20 # Η συνάρτηση σαρώνει τις συχνότητες από 500Hz έως 5KHz για να ακουστεί ήχος συναγερμού
21 def alarm():
22     for i in range(500, 5001, 20): #Σταδιακό ανέβασμα από 200 - 5000
23         tone(i, .001) #Διάρκεια του κάθε τόνου 1msec
24
25 LED.value(0) #Αρχικά στο ξεκίνημα το LED είναι σβηστό
26 t1 = time.ticks_ms() #Κράτησε τον τρέχοντα χρόνο του χρονιστή σε msecs
27
28 # Η συνάρτηση αναβοσβήνει ασύγχρονα το λευκό LED με περίοδο 1sec
29 def blink(en):
30     global led_state, t1
31     if en: #Αν είναι ενεργοποιημένο
32         if time.ticks_ms() - t1 > 500: # Πέρασε ο χρονιστής το 0,5 sec σε σχέση με πριν;
33             t1 = time.ticks_ms() # Αν ναι ξανακράτα τον νέο χρόνο
34             if not LED.value(): #Αν το LED είναι σβηστό
35                 LED.value(1) #Αναψέ το
36             else: #Διαφορετικά
37                 LED.value(0) #Σβήστο
38     else: #Δεν είναι ενεργοποιημένο
39         LED.value(0) #Σβήσε το LED
40
41 # Κυρίως πρόγραμμα
42 # Το PIR βγάζει ψηφιακή έξοδο και αν ανιχνεύσει κίνηση παραμένει σε λογικό '1' για 2 με 3 sec
43 while(True): # Για πάντα
44     s = PIR.value() # Διάβασε τιμή PIR να δεις αν υπάρχει κίνηση.
45     #print(">", s) # Debug
46     if s: # Αν είναι 1 (True)
47         alarm() # Ήχισε συναγερμό
48         blink(True) # Ενεργοποίησε αναβόσβημα του LED
49     else: # Διαφορετικά δεν υπάρχει κίνηση
50         blink(False) # Σταμάτησε το αναβόσβημα του LED
51         time.sleep(.5) # Περίμενε μισό δευτερόλεπτο και ξαναέλεγε το PIR

```

Το συγκεκριμένο PIR έχει εμβέλεια ανίχνευσης 5-8 μέτρα.

Άσκηση 17

Επέκταση της άσκησης 16. Σύστημα συναγερμού

Στο προηγούμενο κύκλωμα θα προσθέσουμε ένα κόκκινο LED τοποθετημένο στη breadboard και ένα push button. Ο κώδικας θα τροποποιηθεί ώστε να υπάρχει η δυνατότητα ενεργοποίησης ή απενεργοποίησης του συναγερμού με το πάτημα του κουμπιού. Επιπλέον το κόκκινο LED θα αναβοσβήνει με διαφορετική περίοδο από το άσπρο, όσο ο συναγερμός είναι ενεργοποιημένος.

Τι θα χρειαστούμε

1. Μικροελεγκτή **ARD:icon II**.
2. Άρθρωμα αισθητήρα P.I.R. **DJS19** (Το kit περιέχει 2 τεμάχια).
3. Άρθρωμα λευκού LED **DJX01**.
4. Άρθρωμα button **DJS09**.
5. Άρθρωμα **EXP-AJ11**.
6. Breadboard.
7. Δύο καλώδια **Dupont F/M** pin.
8. Ένα κόκκινο **LED**.

9. Έναν αντιστάτη 220Ω. (Κόκκινο – Κόκκινο -Μαύρο - Καφέ)
10. Τέσσερα καλώδια RJ12 για ESP32 (μαύρο χρώμα) ή τροποποιημένα.
11. Τουβλάκια και βάση στήριξης.
12. Καλώδιο USB για προγραμματισμό ESP32 και παροχή ρεύματος (Άσπρο type 'C').

Συνδεσμολογία

Τοποθετούμε τα εξαρτήματα και τα συνδέουμε ως εξής:

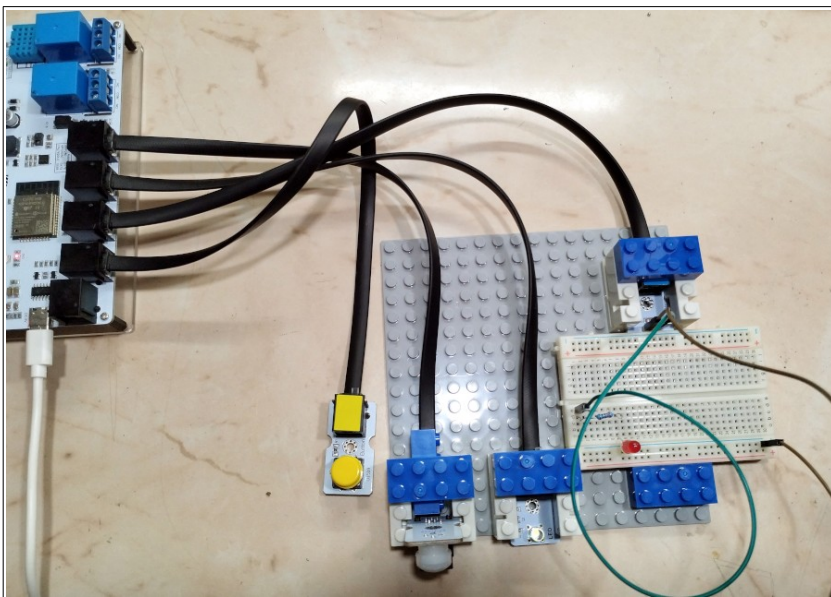
Το PIR στο IO4 του ARD:iconII.

Το λευκό LED στο IO26.

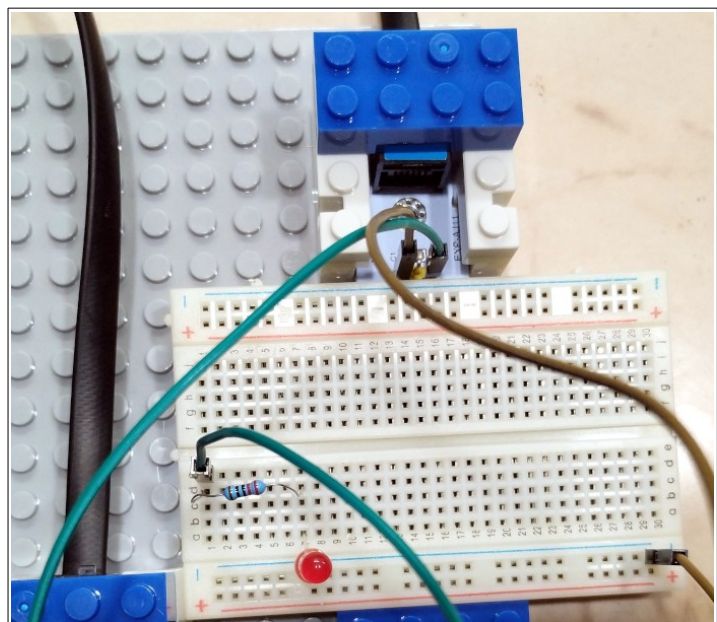
Θα χρησιμοποιήσουμε τον ενσωματωμένο βομβητή της πλακέτας ο οποίος συνδέεται στο IO5.

Το Push Button στο IO34 του ARD:iconII.

Το EXP-AJ11 στο IO32 του ARD:iconII.



Συνδέστε τα δύο καλώδια Dupont F/M σύμφωνα με την διπλανή εικόνα. Το θηλυκό καφέ συνδέεται στο G του EXP-AJ11 και το θηλυκό πράσινο στο S του EXP-AJ11. Από την πλευρά της Breadboard το αρσενικό καφέ πάει στο μπλε (-) και το αρσενικό πράσινο στην οπή e1. Στην d1 συνδέεται ο ένας ακροδέκτης του αντιστάτη και στο d7 ο άλλος ακροδέκτης. Στο a7 συνδέεται ο μακρύς ακροδέκτης του Led και ο κοντός συνδέεται από κάτω στο μπλε (-). Η συνδεσμολογία καθώς και τα χρώματα των καλωδίων είναι ενδεικτικά. Μπορείτε να χρησιμοποιήσετε οποιαδήποτε συνδεσμολογία στην breadboard αρκεί να τηρείτε το θεωρητικό κύκλωμα.



Προγραμματισμός

Δεν απαιτούνται να γίνουν import επιπλέον βιβλιοθήκες πέραν των ενσωματωμένων.

Γράφουμε τον κώδικα στο Thonny ή στο VS Code.

```

1 import time
2 from machine import Pin
3
4 PIR = Pin(4, Pin.IN, Pin.PULL_UP) #PULL_UP ώστε αν κοπεί το καλώδιο του PIR να χτυπάει πάντα
5 BEEPER = Pin(5, Pin.OUT) #Ο ενσωματωμένος βομβητής της πλακέτας συνδέεται στο pin IO5
6 WLED = Pin(26, Pin.OUT) #Το λευκό LED συνδέεται στο pin IO26
7 RLED = Pin(32, Pin.OUT) #Το κόκκινο LED συνδέεται στο pin IO32
8 BUTTON = Pin(34, Pin.IN, Pin.PULL_UP) #PULL_UP
9
10 Alarm_EN = False #Flag για ενεργοποίηση συναγερμού
11
12 # Η συνάρτηση παράγει ένα τόνο συχνότητας freq και διάρκειας dur
13 def tone(freq = 500, dur = .5): #Προκαθορισμένες τιμές
14     #Υπολογισμοί
15     period = 1.0 / freq #Περίοδος
16     half_per = int((period / 2) * 1000000) #Ημιπερίοδος
17     times = int(dur / period) #Αριθμός κύκλων
18     for i in range(times):
19         BEEPER.value(1) #Beeper ενεργό
20         time.sleep_us(half_per) #Περιμένει τον χρόνο της ημιπεριόδου
21         BEEPER.value(0) #Beeper ανενεργό
22         time.sleep_us(half_per) #Περιμένει τον χρόνο της ημιπεριόδου
23
24 # Η συνάρτηση σαρώνει τις συχνότητες από 500Hz έως 5KHz για να ακουστεί ήχος συναγερμού
25 def alarm():
26     for i in range(500, 5001, 20): #Σταδιακό ανέβασμα από 200 - 5000
27         tone(i, .001) #Διάρκεια του κάθε τόνου 1msec
28
29 # Ενέργεια αν πατηθεί το button
30 def onClick():
31     global Alarm_EN
32     if not Alarm_EN:
33         Alarm_EN = True
34         tone(1000, .2)
35     else:
36         Alarm_EN = False
37         tone(500, .1)
38
39 Debounce = 30 #msec Χρόνος αναπήδησης επαφής
40 validclick = False
41 timer = 0
42 downtime = 0
43
44 # Καλείται συνέχεια από το κυρίως πρόγραμμα και ελέγχει αν πατήθηκε το button
45 def chkButton():
46     global validclick, timer, downtime
47     timer = time.ticks_ms() #Κράτησε τον χρόνο συστήματος
48     state = False #Δεν πατήθηκε
49     if not BUTTON.value(): #if BUTTON.value() == 0
50         state = True #Πατήθηκε το κουμπί
51     #----- Πατημένο -----
52     #Πατήθηκε τώρα για πρώτη φορά και ο χρόνος δεν μετράει
53     if state and downtime == 0:
54         downtime = timer #Αρχισε να μετράς τον χρόνο που είναι πατημένο - downtime
55     #Παραμένει πατημένο και έλεγξε αν πέρασε ο χρόνος debounce και δεν έχει ενεργοποιηθεί valid click
56     elif state and not validclick and (timer - downtime) > Debounce:
57         validclick = True #Να μην ξαναμπείς εδώ
58         onClick() #Κάλεσε συνάρτηση εξυπηρέτησης
59     #----- Ελεύθερο -----
60     #Αλλιώς αν ήταν πατημένο και τώρα το άφησε
61     elif not state:
62         downtime = 0 #Επαναφορά
63         validclick = False #Επαναφορά για την επόμενη φορά
64
65 #Κλάση για αναβόσβημα των LED
66 class Blink:
67     def __init__(self, Led, Dur):
68         self.Led = Led #IO pin του LED
69         self.Dur = Dur #Διάρκεια ημιπεριόδου
70         self.t = time.ticks_ms() #Χρονιστής
71         self.en = False #Ενεργοποιημένο
72
73     def blink(self):
74         if self.en: #Αν είναι ενεργοποιημένο
75             if time.ticks_ms() - self.t > self.Dur: # Πέρασε ο χρονιστής το 0,5 sec σε σχέση με πριν;
76                 self.t = time.ticks_ms() # Αν ναι ξανακράτα τον νέο χρόνο
77                 if not self.Led.value(): #Αν το LED είναι σβηστό
78                     self.Led.value(1) #Αναψέ το
79                 else: #Διαφορετικά
80                     self.Led.value(0) #Σβήστο
81             else: #Δεν είναι ενεργοποιημένο
82                 self.Led.value(0) #Σβήστο το LED
83
84 #Ετιγμιότυπα για τα 2 LED
85 blinkRed = Blink(RLED, 100) #Κόκκινο με περίοδο 200msec
86 blinkRed.en = False #Αρχικά δεν αναβοσβήνει
87 blinkWhite = Blink(WLED, 500) #Λευκό με περίοδο 1sec
88 blinkWhite.en = False #Αρχικά σβηστό
89

```



```

90 # Κυρίως πρόγραμμα - εκτελείται συνέχεια
91 # Το PIR βγάζει ψηφιακή έξοδο και αν ανιχνεύσει κίνηση παραμένει σε λογικό '1' για 2 με 3 sec
92 while(True): # Για πάντα
93     chkButton() #Ελέγχος αν πατήθηκε το Button
94     blinkRed.blink() #Κάλεσε μέθοδο για αναβόσβημα του κόκκινου
95     blinkWhite.blink() #Κάλεσε μέθοδο για αναβόσβημα του λευκού
96     if Alarm_EN: #Αν έχει ενεργοποιηθεί ο συναγερμός
97         blinkRed.en = True #Να αναβοσβήνει το κόκκινο LED
98     else: #Διαφορετικά
99         blinkRed.en = False #Να σταματήσει τις αναλαμπές
100     s = PIR.value() # Διάβασε τιμή PIR να δεις αν υπάρχει κίνηση.
101     if s and Alarm_EN: # Αν είναι 1 (True) ανιχνεύθηκε κίνηση και έχει ενεργοποιηθεί
102         alarm() # Ήχους συναγερμό
103         blinkWhite.en = True # Ενεργοποίησε αναβόσβημα του λευκού LED
104     else: # Διαφορετικά δεν υπάρχει κίνηση ή είναι απενεργοποιημένο
105         blinkWhite.en = False # Σταμάτησε το αναβόσβημα του λευκού LED
106         time.sleep(.02) # Περίμενε 20msec και ξαναέλεγε το PIR

```

Βίντεο επίδειξης στο youtube: <https://youtu.be/adhmqCZgrWU>

Άσκηση 18

Έξυπνο σπίτι - Ελεγκτής IOT

Σ' αυτή την άσκηση θα δημιουργήσουμε έναν κόμβο **I.O.T.** (Internet Of Things). Οι κόμβοι IOT είναι συσκευές όπως διακόπτες, πρίζες, λάμπες, αισθητήρες κλιματικών μεγεθών κλπ. Όλες αυτές οι συσκευές επικοινωνούν μέσω διαδικτύου με ένα κεντρικό διακομιστή στον οποίο στέλνουν δεδομένα ή δέχονται εντολές, ώστε να πραγματοποιήσουν ένα **Έξυπνο Σπίτι**.

Αν και πολλές εταιρίες έχουν αναπτύξει τα δικά τους οικοσυστήματα όπως Google Home, Amazon Alexa, Apple Homekit κ.α. υπάρχουν και πολλές εναλλακτικές λύσεις ανοιχτού κώδικα με απεριόριστες δυνατότητες.

Εδώ θα χρησιμοποιήσουμε το πρωτόκολλο **mqtt** για επικοινωνία και ανταλλαγή μηνυμάτων μεταξύ των συσκευών και την πλατφόρμα προγραμματισμού **NodeRed** για προγραμματισμό του σεναρίου υλοποίησης του έξυπνου σπιτιού. Στα **παραρτήματα Α' και Β'** υπάρχουν οδηγίες εγκατάστασης και δοκιμής του **Mosquitto MQTT broker** και του **Node Red**.

Για λόγους απλότητας στο dashboard του υπολογιστή ή του smartphone θα βλέπουμε την θερμοκρασία και την υγρασία του δωματίου και θα μπορούμε να χειριστούμε με ένα διακόπτη την ενεργοποίηση ή απενεργοποίηση μιας συσκευής π.χ. ένα φως. Επειδή η πλακέτα έχει πάρα πολλές δυνατότητες εσείς μπορείτε να επεκτείνετε τις λειτουργίες.

Τι θα χρειαστούμε

1. Μικροελεγκτή **ARD:icon II**.
2. Άρθρωμα button **DJS09**.
3. Ένα καλώδιο RJ12 για ESP32 (μαύρο χρώμα) ή τροποποιημένα.
4. Καλώδιο USB για προγραμματισμό ESP32 και παροχή ρεύματος (Άσπρο type 'C').

Συνδεσμολογία

Η συνδεσμολογία είναι απλή. Συνδέουμε το άρθρωμα DJS09 με το καλώδιο το οποίο από το άλλο άκρο συνδέεται με το IO34 του ARD:iconII.

Προγραμματισμός

Δεν απαιτούνται να γίνουν import επιπλέον βιβλιοθήκες πέραν των ενσωματωμένων.

Γράφουμε τον κώδικα στο Thonny ή στο VS Code. Θα χρειαστούμε δύο αρχεία. Το **boot.py** με τις πληροφορίες σύνδεσης στο τοπικό δίκτυο WiFi και το **main.py** με όλες τις λειτουργίες επικοινωνίας με τον mqtt broker.

Αρχείο boot.py

```

1 import time, network, gc, esp
2
3 esp.osdebug(None)
4 gc.collect()
5 ssid = 'SSID' #Το SSID του τοπικού δικτύου WiFi
6 password = 'WiFi_KEY' #Ο κωδικός του WiFi
7 station = network.WLAN(network.STA_IF) #θα λειτουργήσει σαν σταθμός ώστε να συνδεθεί στο AP
8
9 station.active(True) #Ενεργοποίηση
10 station.connect(ssid, password) #Σύνδεση
11
12 while station.isconnected() == False: #Περίμενε μέχρι να συνδεθεί
13     pass
14
15 print('Connection successful')
16 print(station.ifconfig())
17
18 time.sleep(1) # Περίμενε 1sec μέχρι να γίνει η σύνδεση και μετά συνέχισε

```

Αρχείο main.py

```

1 # ===== Κυρίως πρόγραμμα εκτελείται μετά το boot.py =====
2 import network, gc, time, binascii, ntptime, dht
3 from machine import Pin, unique_id
4 from umqtt.simple import MQTTClient
5
6 # Ορισμοί ακροδεκτών και αισθητήρων
7 BUTTON1 = Pin(34, Pin.IN, Pin.PULL_UP) #PULL_UP
8 Relay1 = Pin(16, Pin.OUT)
9 sensor1 = dht.DHT11(Pin(33))
10
11 # Στοιχεία σύνδεσης στον μεσίτη MQTT
12 mqtt_server = '192.168.42.30' #Διεύθυνση IP ή όνομα π.χ. mqtt1.example.sch.gr
13 mqtt_user = 'mqttuser' #Προαιρετικά αν απαιτείται πιστοποίηση
14 mqtt_pass = '123456'
15 client_id = binascii.hexlify(unique_id()) #Το όνομα client αν θέλω να το χρησιμοποιήσω στα topics
16 place = "place1" #Τοποθεσία της συσκευής π.χ. bathroom, bedroom, livingroom, wcl κλπ.
17 print("Ταυτότητα: %s" % (client_id.decode("utf-8"))) #Εμφάνιση ταυτότητας στο τερματικό
18
19 # Συγχρονισμός ώρας με ntp server
20 ntptime.host = "europe.pool.ntp.org"
21 ntptime.timeout = 3
22 ntptime.settime() # Συγχρονισμός
23 UTC_OFFSET = 3 # Ελλάδα +3 ώρες θερινή
24
25 # --- Ορισμός των topics ---
26 # Θα χρησιμοποιήσουμε για κάθε συσκευή 3 topics.
27 # Ένα sub στο οποίο κάνει συνδρομή η συσκευή και ακούει συνεχώς για δημοσίευση μηνυμάτων και
28 # δύο pub για δημοσίευση μηνυμάτων προς τον μεσίτη. Το τελευταίο (tele) είναι για αποστολή τηλεμετρίας και
29 # δημοσιεύει σε τακτά χρονικά διαστήματα π.χ. κάθε 1 λεπτό
30 topic1_sub = b'cmd/' + place + '/POWER1' # Συνδρομή
31 topic1_pub = b'stat/' + place + '/POWER1' # Δημοσίευση αν γίνει αλλαγή κατάστασης
32 topic2_pub = b'tele/' + place + '/STATE' # Δημοσίευση κάθε 1 δευτερόλεπτα
33 # Ορισμός περιόδου tele
34 TELE = 1 # Λεπτά
35 TelePeriod = TELE * 60 * 1000 # secs * msecs
36 Resync = 3 # Ώρες. Επανασυγχρονισμός με ntp server
37 NTP_SYNC = Resync * 60 // TELE
38
39 # Δημοσιεύει μήνυμα stat ώστε να ενημερώσει για την κατάσταση του διακόπτη
40 def stat_pub(topic, message):
41     client.publish(topic, message, retain=True) # Πρέπει να είναι Retain
42     print("Stat:", message) # Debug
43
44 # Ενεργοποιεί ή απενεργοποιεί την συσκευή
45 def set_relay(state):
46     if state:
47         print("Η συσκευή ενεργοποιήθηκε") # Debug
48         stat_pub(topic1_pub, b'ON')
49         Relay1.value(True) # Κλείσιμο επαφής
50     else:
51         print("Η συσκευή απενεργοποιήθηκε") # Debug
52         stat_pub(topic1_pub, b'OFF')
53         Relay1.value(False) # Άνοιγμα επαφής
54
55 # Ενεργεί αν πατηθεί το button
56 def onClick():
57     if not Relay1.value(): # Αν είναι OFF
58         set_relay(True) # Να γίνει ON
59     else: # Αλλιώς
60         set_relay(False) # Να γίνει OFF
61
62 # Καθολικές μεταβλητές για chkButton
63 Debounce = 30 #msec Χρόνος αναπήδησης επαφής
64 validclick = False
65 timer = 0
66 downtime = 0
67
68 # Καλείται συνέχεια από το κυρίως πρόγραμμα και ελέγχει αν πατήθηκε το button
69 def chkButton():
70     global validclick, timer, downtime
71     timer = time.ticks_ms() #Κράτησε τον χρόνο συστήματος
72     state = False #Δεν πατήθηκε
73     if not BUTTON1.value(): #if BUTTON1.value() == 0
74         state = True #Πατήθηκε το κουμπί
75     #----- Πατημένο -----
76     #Πατήθηκε τώρα για πρώτη φορά και ο χρόνος δεν μετράει
77     if state and downtime == 0:
78         downtime = timer #Άρχισε να μετράς τον χρόνο που είναι πατημένο - downtime
79         #Παράμεινε πατημένο και έλεγξε αν πέρασε ο χρόνος debounce και δεν έχει ενεργοποιηθεί valid click
80         elif state and not validclick and (timer - downtime) > Debounce:
81             validclick = True #Να μην ξαναμπείς εδώ
82             onClick() #Κάλεσε συνάρτηση εξυπηρέτησης
83     #----- Ελεύθερο -----
84     #Αλλιώς αν ήταν πατημένο και τώρα το άφησε
85     elif not state:

```

```

86     downtime = 0 #Επανάφορά
87     validclick = False #Επανάφορά για την επόμενη φορά
88
89 # Εξυπηρέτηση δημοσιεύσεων στα topics που έγινε συνδρομή
90 def sub_cb(topic, msg):
91     #print((topic, msg)) # Debug
92     if topic == topic1_sub: # Αν το topic μας αφορά π.χ. cmnd/place1/POWER1
93         if msg == b'ON': # Αν το μήνυμα είναι ON
94             set_relay(True) # Ενεργοποίηση συσκευής
95         elif msg == b'OFF': # Αν το μήνυμα είναι OFF
96             set_relay(False) # Απενεργοποίηση συσκευής
97
98 # Σύνδεση στον broker και συνδρομή στα topics
99 def connect_and_subscribe():
100     client = MQTTClient(client_id, mqtt_server, user=mqtt_user, password=mqtt_pass)
101     client.set_callback(sub_cb) # Συνάρτηση callback. Καλείται αν λάβει μήνυμα από μεσίτη
102     try:
103         client.connect() # Σύνδεση
104         client.subscribe(topic1_sub) # Συνδρομή
105         print('Συνδέθηκε στον μεσίτη MQTT %s και έγινε συνδρομή στα θέματα: %s' % (mqtt_server, topic1_sub.decode("utf-8")))
106         return client
107     except:
108         print('Πρόβλημα στη σύνδεση')
109
110 # Διαβάζει την ημερομηνία και την ώρα συστήματος
111 def get_time():
112     t = time.gmtime(time.time() + UTC_OFFSET * 3600)
113     ts = str(t[2]) + '/' + str(t[1]) + '/' + str(t[0]) + '-' + str(t[3]) + ':' + str(t[4]) + ':' + str(t[5])
114     return ts
115
116 Relay1.value(False) # Αρχικά ο διακόπτης θα είναι off
117
118 # Προσπάθεια σύνδεσης στον broker
119 client = connect_and_subscribe()
120 if client == None:
121     print('Προσπάθεια επανασύνδεσης ...')
122     time.sleep(5)
123     machine.reset()
124
125 print('Ωρα συστήματος:', get_time()) # Debug
126
127 tele_time = 0 # Μετρητής περιόδου tele
128 measure_time = 0 # Μετρητής χρόνου δειγματοληψίας μετρήσεων
129 ntp_sync = NTP_SYNC
130 while True: # Για πάντα
131     client.check_msg() # Έλεγξε για μήνυμα που αφορά την συνδρομή συνεχώς
132     chkButton() # Έλεγξε μήπως πατήθηκε το button
133     if time.ticks_ms() - measure_time > 5000: # Κάθε 5 secs
134         measure_time = time.ticks_ms() # Μέτρα από την αρχή
135         sensor1.measure() # Πάρε τιμές θερμοκρασίας και υγρασίας
136     if time.ticks_ms() - tele_time > TelePeriod: # Κάθε 1 - 5 λεπτά στείλε τηλεμετρία
137         tele_time = time.ticks_ms() # Μέτρα από την αρχή
138         if Relay1.value(): # Σε τι κατάσταση είναι η συσκευή
139             t = 'POWER1:ON' # Σε λειτουργία
140         else:
141             t = 'POWER1:OFF' # Σβηστή
142         # Ετοίμασε μήνυμα csv χωρισμένο με ';'
143         ts = get_time() + ';' + str(sensor1.temperature()) + ';' + str(sensor1.humidity()) + ';' + t
144         client.publish(topic2_pub, t.encode()) # Δημοσίευσε το tele
145         print("Tele: ", t.encode()) # Debug
146         if ntp_sync <= 0:
147             ntp_sync = NTP_SYNC
148             print("Επανασυγχρονισμός NTP") # Debug
149             ntp.time.settime() # Συγχρονισμός
150         else:
151             ntp_sync -= 1

```

Το πρόγραμμα συνδέεται στον τοπικό μεσίτη mqtt (οδηγίες εγκατάστασης και χρήσης στο Παράρτημα 1), και χρησιμοποιεί τρία διαφορετικά topics :

1. Το **cmnd** στο οποίο η συσκευή κάνει συνδρομή και περιμένει να ακούσει μια εντολή αλλαγής της κατάστασης του διακόπτη από τον μεσίτη π.χ. 'ON' ή 'OFF'
2. Το **stat** στο οποίο η συσκευή δημοσιεύει στον μεσίτη όταν αλλάζει η κατάσταση του διακόπτη. Αν κάποιος πιάσει το button χειροκίνητα και ανάψει το φως τότε πρέπει να ενημερωθεί ο μεσίτης.
3. Το **tele** στο οποίο η συσκευή δημοσιεύει στον μεσίτη σε τακτά χρονικά διαστήματα π.χ. κάθε 5 λεπτά, την κατάσταση του διακόπτη, ώρα, θερμοκρασία και ποσοστό υγρασίας.

Δοκιμή

Εκτελέστε το πρόγραμμα με **ctrl + D** και δείτε κάτω στο κέλυφος ότι όλα πάνε καλά.

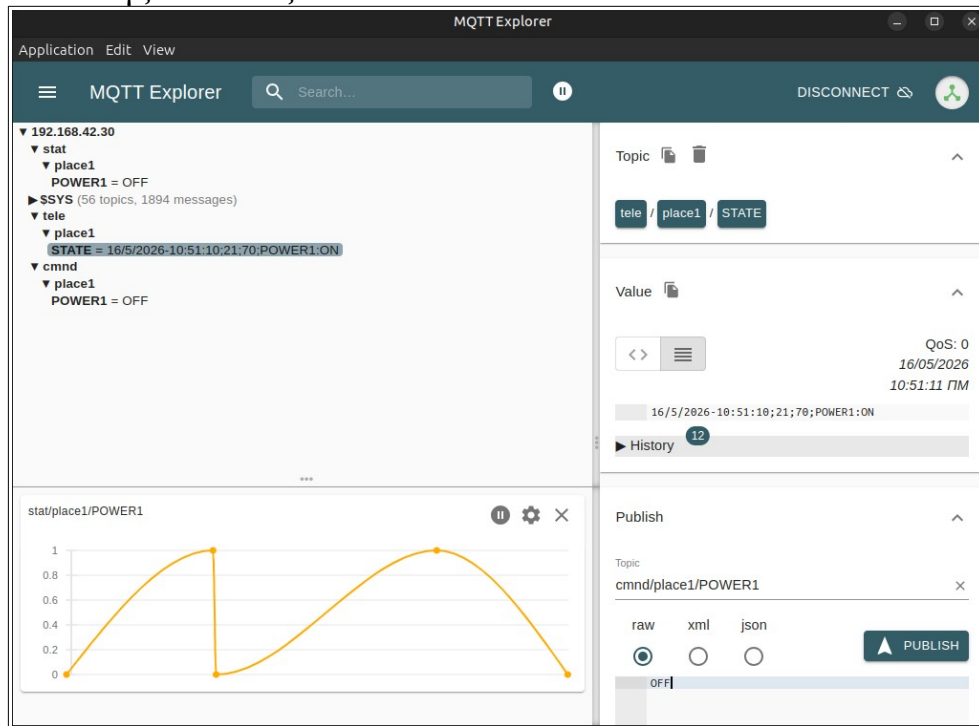
Ανοίξτε το διαγνωστικό πρόγραμμα **MQTT Explorer** και συνδεθείτε στον τοπικό broker.

Αρχικά θα δείτε ότι στο δέντρο εκτός από το **\$SYS** υπάρχει το topic **stat** το οποίο είναι **Retain** (Μόνιμο). Μετά από 1 – 5 λεπτά έρχεται και το **tele**.

Πατήστε το button και δείτε αν αλλάζει η κατάσταση του **stat**.

Τέλος στην δεξιά στήλη στο πεδίο **Publish** στην θέση topic γράψτε **cmnd/place1/POWER1** και από κάτω επιλέξτε raw και πληκτρολογήστε **ON**. Τέλος πατήστε το κουμπί PUBLISH.

Παρατηρήστε ότι προστέθηκε και ο κλάδος **cmnd** στο δέντρο του broker. Αν το ρελέ όπλισε γράψτε **OFF** και ελέγξτε αν άνοιξε.



Αν όλα πήγαν καλά θα προχωρήσουμε στο επόμενο βήμα υλοποίησης της λογικής αυτοματισμού με το **Node Red**. Οδηγίες εγκατάστασης στο **Παράρτημα Β'**.

Εισαγωγή στο NodeRed

Τα βασικά

Συνδεόμαστε στον editor δίνοντας http://local_server_ip:1880.

Αρχικά πατάμε πάνω δεξιά στις τρεις γραμμές – Flows – Edit και αλλάζουμε το όνομα σε κάτι διαφορετικό από Flow1. Βρίσκουμε το Node **mqtt in** από την ομάδα **network** και το βάζουμε μέσα στο Flow. Πατάμε διπλό κλικ για να διορθώσουμε.

Αρχικά ζητάει να δηλώσουμε mqtt broker όπου δίνουμε ένα όνομα π.χ. Local Broker και διεύθυνση IP. Στο tab Security δίνουμε τα διαπιστευτήρια σύνδεσης και μετά **Update**.

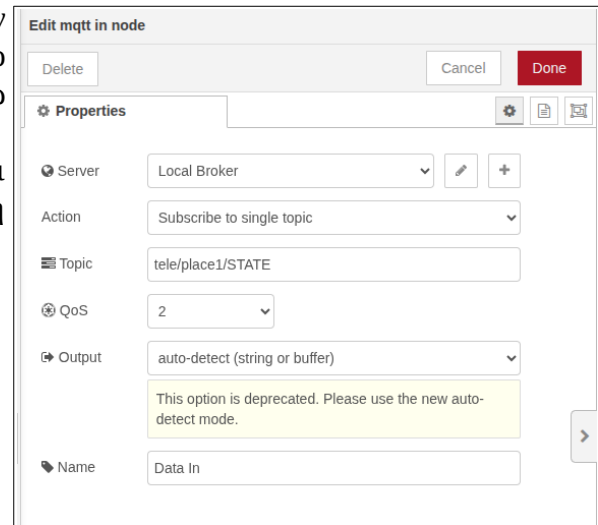
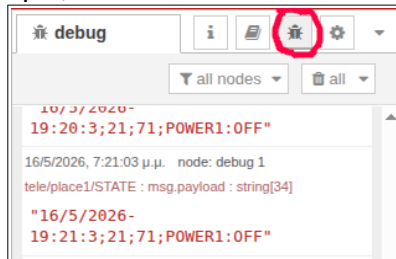
The screenshot shows the 'Properties' dialog for the 'mqtt in' node. The 'Name' field is 'Local Broker'. The 'Connection' tab is selected. The 'Server' field is '192.168.42.30' and the 'Port' is '1883'. The 'Connect automatically' checkbox is checked. The 'Protocol' is 'MQTT V3.1.1'. The 'Client ID' is 'Leave blank for auto generated'. The 'Keep Alive' is '60'. The 'Session' checkbox 'Use clean session' is checked.

The screenshot shows the 'Properties' dialog for the 'mqtt in' node. The 'Name' field is 'Local Broker'. The 'Security' tab is selected. The 'Username' field is 'mqttuser' and the 'Password' field is filled with dots. The 'Update' button is highlighted in red.

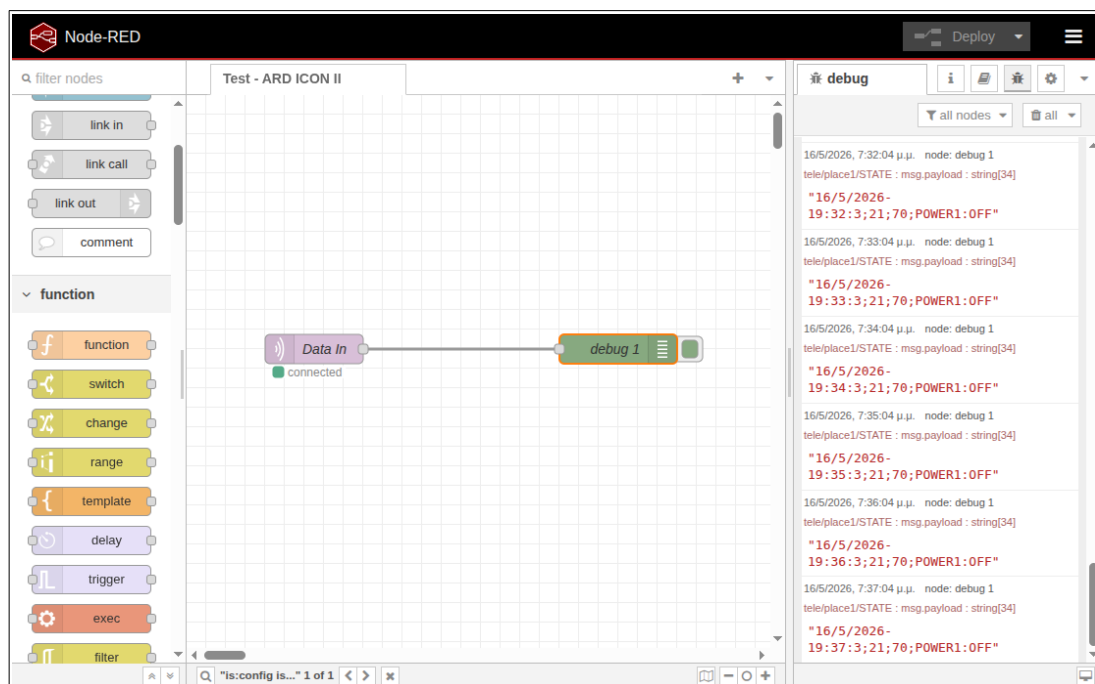
Τώρα συμπληρώνουμε το node mqtt in όπου επιλέγουμε τον broker που ορίσαμε στο προηγούμενο βήμα, προσθέτουμε το topic **tele/place1/STATE** , αλλάζουμε το Output σε **auto-detect (string or buffer)** και δίνουμε και ένα όνομα π.χ. Data In. Πατάμε το κόκκινο κουμπί Done.

Μετά προσθέτουμε και ένα node **debug** από την ομάδα **common**. Συνδέουμε τα δύο nodes και πατάμε το κόκκινο κουμπί **Deploy** πάνω δεξιά ώστε να αποθηκεύσει τις αλλαγές.

Τώρα πατάμε το κουμπί debug στην δεξιά στήλη και από κάτω βλέπουμε τα μηνύματα tele που στέλνει η μονάδα μας.



Αν θέλουμε να απενεργοποιήσουμε προσωρινά το debugging γι' αυτό το node αρκεί να πατήσουμε το πράσινο κουμπί και αυτό θα μπει μέσα. Μετά πάλι Deploy.



Πάντα όταν στο tab του ενεργού Flow βλέπουμε μια γαλάζια κουκίδα και σε κάποια nodes το ίδιο, σημαίνει ότι έγιναν αλλαγές και πρέπει να γίνει deploy.

Αν βλέπετε την παραπάνω εικόνα καταφέρατε να πάρετε τα δεδομένα από το ARD ICON II και στη συνέχεια να έχετε την δυνατότητα επεξεργασίας μέσα από το NodeRed.

Επεξεργασία και παρουσίαση των δεδομένων

Τώρα θα επεξεργαστούμε το μήνυμα tele ώστε να εμφανίζει στο dashboard όργανα με τις μετρήσεις. Αρχικά σβήνουμε το debug 1 που βάλαμε πριν πατώντας το πλήκτρο **Delete**.

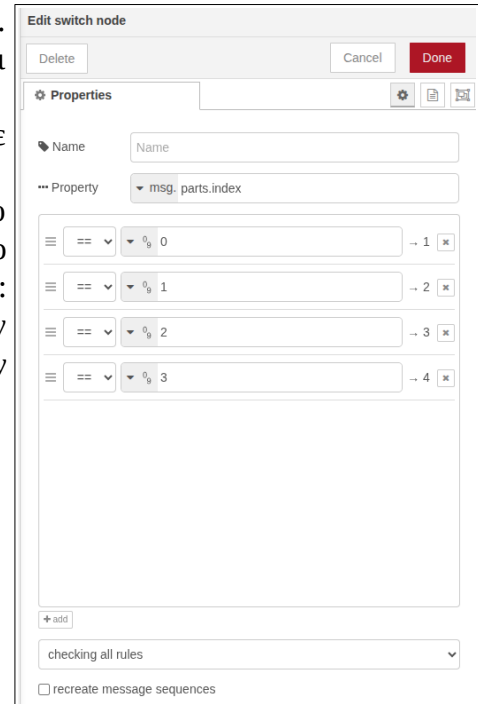
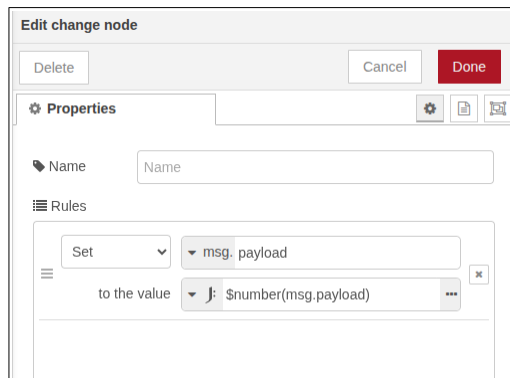
Από ομάδα **sequence** βάζουμε το **split** και το ενώνουμε με το mqtt in. Το διορθώνουμε βάζοντας στο πεδίο **split using** το αλφαριθμητικό (σύμβολο 'az') ; (ελληνικό ερωτηματικό) που είναι ο χαρακτήρας διαχωρισμού csv.

Από την ομάδα **function** προσθέτω το **switch** και το συνδέω με το προηγούμενο split. Το διορθώνω με διπλό κλικ. Στο πεδίο property βάζω **msg.parts.index** και προσθέτω συνθήκες πατώντας το **+add** για να βάλω γραμμές. Η συνθήκες είναι == αλλά διορθώνω ώστε να είναι **αριθμητικό** (σύμβολο '09') και είναι με την σειρά από 0 έως και 3. Μετά Done.

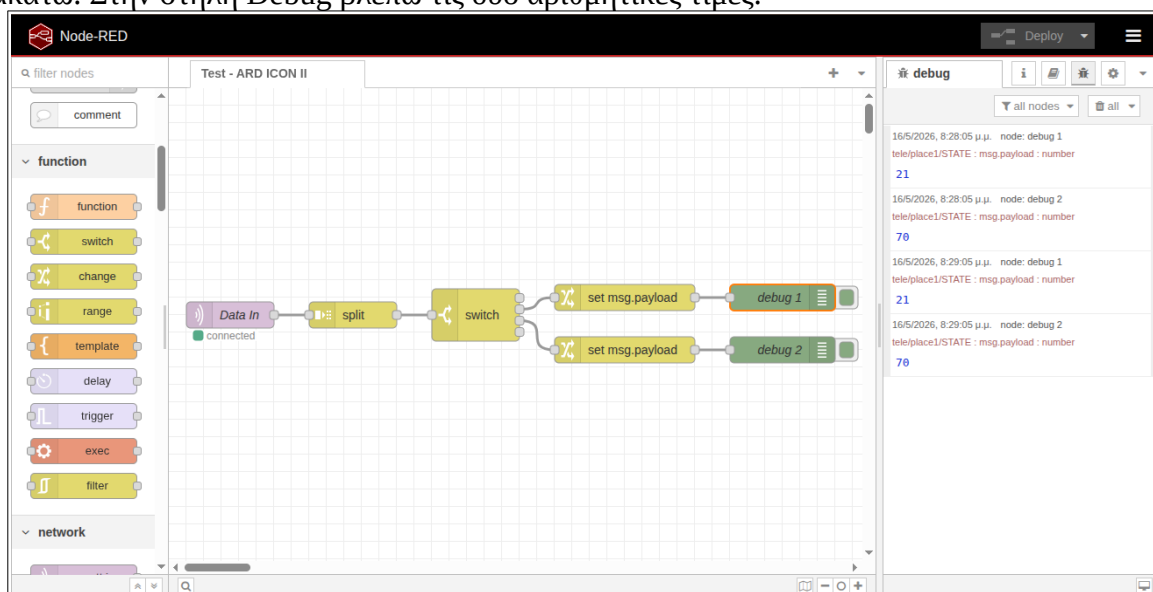
Αυτό θα μας δώσει τέσσερις εξόδους, μία για κάθε πεδίο. Αρχικά θα χρησιμοποιήσουμε την 2η και 3η που είναι θερμοκρασία και υγρασία από το DHT11.

Επειδή τα πεδία είναι αλφαριθμητικά θα τα μετατρέψουμε σε αριθμητικά με δύο nodes **change** από την ομάδα **function**.

Μεταφέρω τον πρώτο και τον συνδέω με την 2η έξοδο του switch. Τον διορθώνω αλλάζοντας το **Change** σε **Set** και στο πεδίο **to the value** αλλάζω σε **expression J** και γράφω μέσα: **\$number(msg.payload)** . Πατάω Done. Αντιγράφω αυτόν τον κόμβο με **Ctrl + C** και τον επικολλώ με **Ctrl + V** και τον συνδέω με την 3η έξοδο του switch.



Τέλος προσθέτω δύο **debug** nodes για να ελέγξω αν όλα πήγαν καλά. Το Flow τώρα είναι όπως παρακάτω. Στην στήλη Debug βλέπω τις δύο αριθμητικές τιμές.



To dashboard

Για να παρουσιάσουμε τα δεδομένα στον χρήστη έχουμε πάρα πολλά όμορφα **controls** τόσο στο dashboard 2 όσο και στο παλαιότερο dashboard 1. Πάμε στην ομάδα **dashboard 2** και τοποθετούμε δύο nodes τύπου **gauge**. Για να ρυθμίσουμε την διάταξη του dashboard πατάμε πάνω δεξιά στο βελάκι (προς τα κάτω) δίπλα από το γρανάζι και επιλέγουμε dashboard 2. Όπως βλέπουμε στο Layout μπορούμε να έχουμε διαφορετικές **σελίδες** και μέσα σε κάθε σελίδα διαφορετικά **groups** και μέσα στο κάθε group τοποθετούνται τα **controls**. Για να αλλάξουμε την διάταξη των controls μπορούμε να τα σύρουμε πάνω κάτω στο group με **drag and drop**. Επίσης στην σελίδα υπάρχει και ο **layout editor** με τον οποίο αλλάζω τα μεγέθη και την θέση των αντικειμένων.

Ακολουθούν οι ρυθμίσεις για τα δύο gauge θερμοκρασίας και υγρασίας. Είναι διαφορετικού τύπου και μπορείτε να αλλάξετε τα χρώματα, τις μονάδες, τις ετικέτες και να τα παραμετροποιήσετε όπως θέλετε.

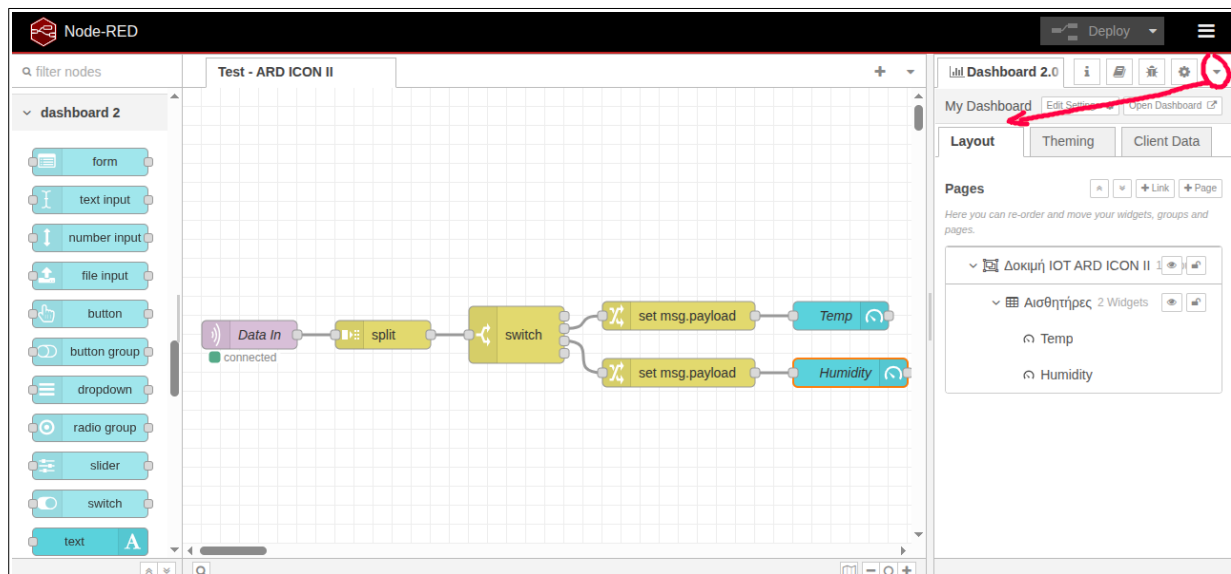
The 'Edit gauge node' dialog for a temperature gauge shows the following settings:

- Name:** Temp
- Group:** [Δοκιμή IOT ARD ICON II] Αισθητήρες
- Size:** 2 x 3
- Type:** Half Gauge
- Style:** Needle
- Value:** msg.payload
- Limits:** min. -10, max. 50
- Segments:** Three segments with values -10, 10, and 20, each with a 'Label' dropdown.
- Labelling:** Label: Θερμοκρασία, Units: °C, Icon: Icon from mdi library, e.g. 'home'

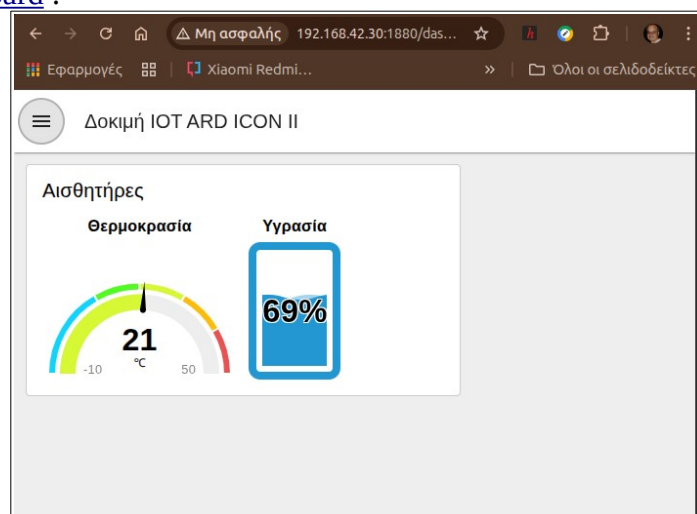
The 'Edit gauge node' dialog for a humidity gauge shows the following settings:

- Name:** Humidity
- Group:** [Δοκιμή IOT ARD ICON II] Αισθητήρες
- Size:** 1 x 3
- Type:** Tank Level
- Value:** msg.payload
- Limits:** min. 0, max. 100
- Segments:** Four segments with values 0, 15, 35, and 50, each with a 'Label' dropdown.
- Labelling:** Label: Υγρασία, Class: segmental CSS class name(s) for widget

Τώρα το Flow έχει τροποποιηθεί όπως στην ακόλουθη εικόνα:



Ο χρήστης μπορεί να δει το **user interface** γράφοντας στον browser την διεύθυνση <http://local-server-ip:1880/dashboard>.

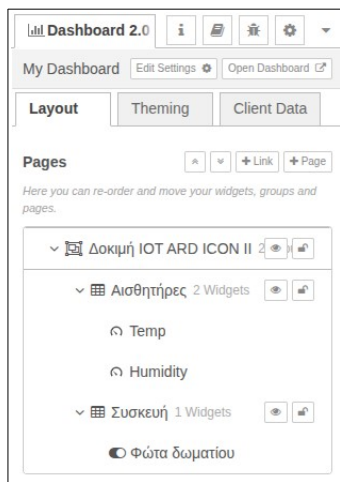


Χειρισμός της συσκευής με διακόπτη

Για να έχουμε την δυνατότητα χειρισμού της συσκευής απομακρυσμένα θα προσθέσουμε έναν ακόμη κόμβο **mqtt in** από την ομάδα **network**. Αυτός ο κόμβος κάνει συνδρομή στο topic **stat** και έχει ρυθμιστεί όπως στην διπλανή εικόνα: Μετά στο dashboard 2 προσθέτουμε μια νέα ομάδα με ονομασία 'Συσκευή' και σ' αυτή θα προσθέσουμε τον διακόπτη.

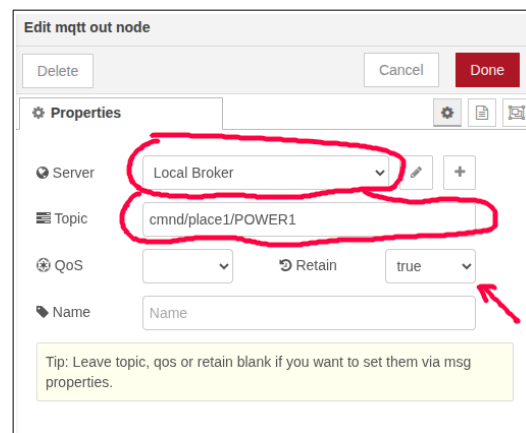
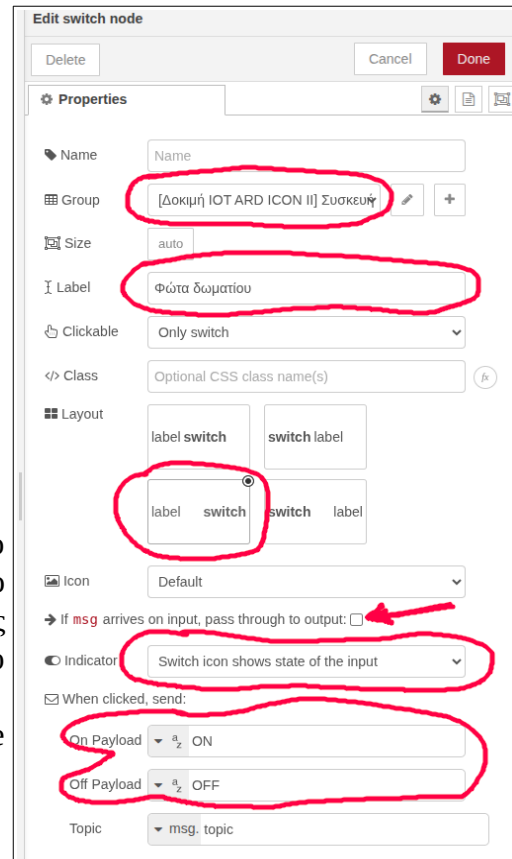
Από την ομάδα **dashboard 2** προσθέτουμε ένα **switch** στο flow και το συνδέουμε με το mqtt in (stat).

Προσοχή στις ρυθμίσεις του διακόπτη. Ότι είναι σημειωμένο με κόκκινο έχει αλλάξει.

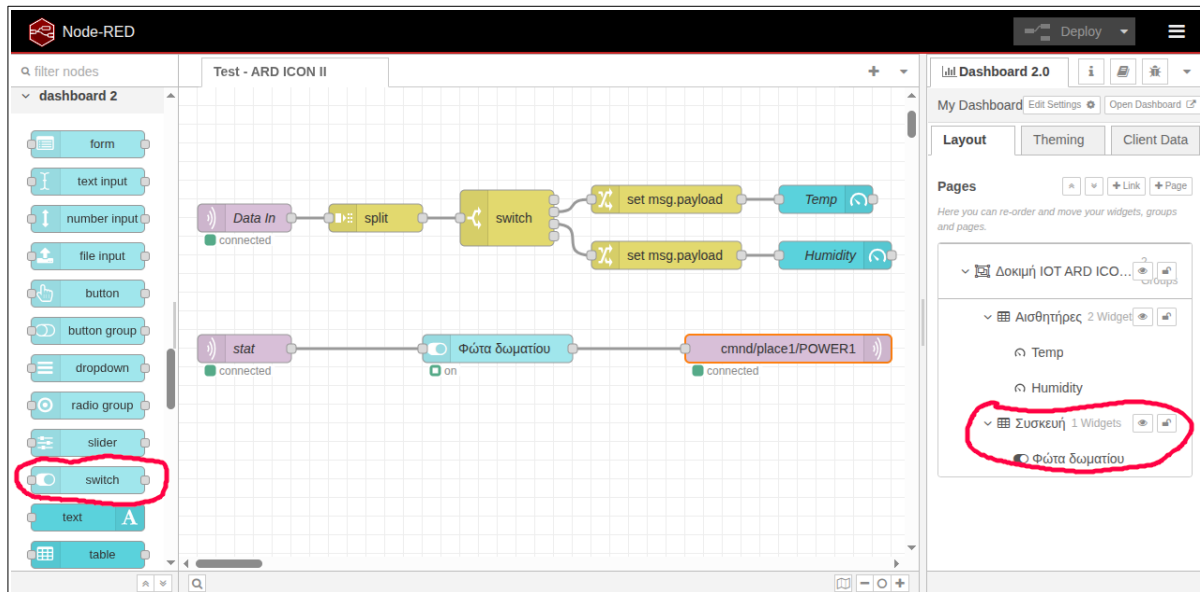


Τέλος πρέπει να προσθέσουμε και έναν κόμβο **mqtt out** από την ομάδα **network**. Μ' αυτόν, το NodeRed θα κάνει **publish** στο topic **cmnd** της συσκευής. Το mqtt out συνδέεται με την έξοδο του διακόπτη.

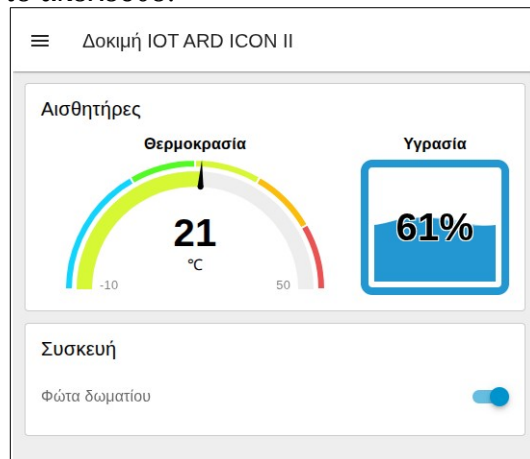
Κάνουμε τις αλλαγές στα πεδία του mqtt out node σύμφωνα με την εικόνα κάτω δεξιά.



Τώρα το τελικό Flow φαίνεται στην επόμενη εικόνα. Φυσικά δεν ξεχνάμε να κάνουμε **Deploy**.



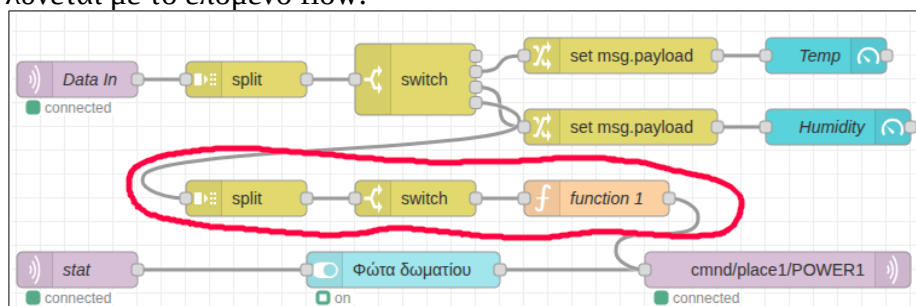
Το περιβάλλον χρήστη είναι το ακόλουθο:



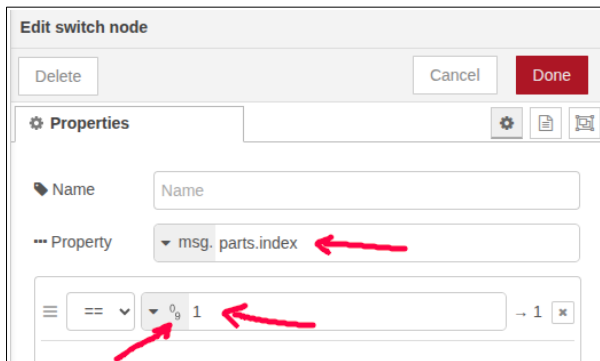
Δοκιμάστε να ανοίξετε και να κλείσετε τον διακόπτη από το dashboard και ελέγξτε αν ανοιγοκλείνει το ρελέ. Μετά πατήστε το button στην συσκευή και ελέγξτε αν ενημερώνεται το dashboard. Ανάψτε το φως από το dashboard και κλείστε για λίγο την συσκευή ARD ICON II. Ανάψτε πάλι το ARD ICON II και παρατηρήστε ότι μόλις συνδεθεί στον mqtt broker θα γυρίσει στην προηγούμενη κατάσταση δηλ. ON λόγω της επιλογής **Retain true** στον κόμβο **mqtt out** του Flow.

Αν κάνουμε το ίδιο από το button της συσκευής δεν θα έχουμε το ίδιο αποτέλεσμα μετά την διακοπή γιατί η αλλαγή που μπήκε από το stat δεν ενημέρωσε το Retain του mqtt out.

Το πρόβλημα λύνεται με το επόμενο flow:



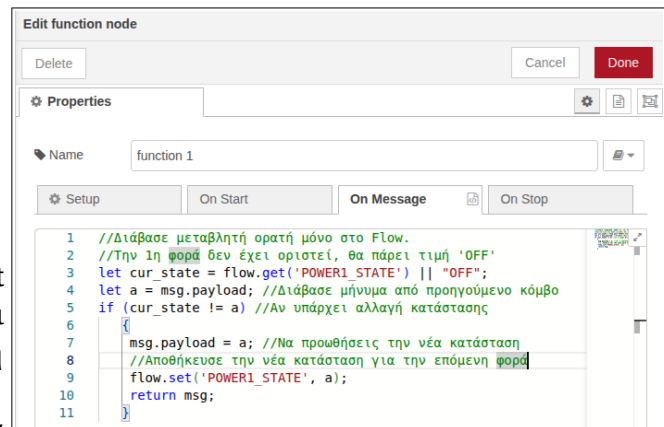
Από το **tele** που λαμβάνεται κάθε 1 λεπτό επιλέγουμε το 4ο μήνυμα που έχει τιμές **'POWER1:ON'** ή **'POWER1:OFF'**. Ο κόμβος **split** διαχωρίζει το μήνυμα με τον χαρακτήρα **:**. Έτσι τώρα έχουμε δύο τμήματα, το πρώτο **'POWER1'** και το δεύτερο **'ON'** ή **'OFF'**. Στη συνέχεια με την **switch** κρατάμε μόνο το **2ο** τμήμα το οποίο μας πληροφορεί για την τρέχουσα κατάσταση.



Το μήνυμα συνδέεται στον κόμβο **mqtt out** ώστε να στείλει εντολή **cmdn** αν υπάρχει αλλαγή από το button της συσκευής ώστε αυτή να γίνει **Retain**.

Αν πατήσουμε το button στην συσκευή θα πρέπει να περιμένουμε τον χρόνο της περιόδου **tele** δηλ. ένα λεπτό ώστε να ενημερωθεί η κατάσταση. Αν μετά σβήσουμε το ARD ICON II και το ανάψουμε πάλι, θα θυμηθεί την προηγούμενη κατάσταση.

Τέλος ακολουθεί η **συνάρτηση** από την ομάδα **function**. Μέσα γράφουμε τον παρακάτω κώδικα σε γλώσσα **Java Script**.



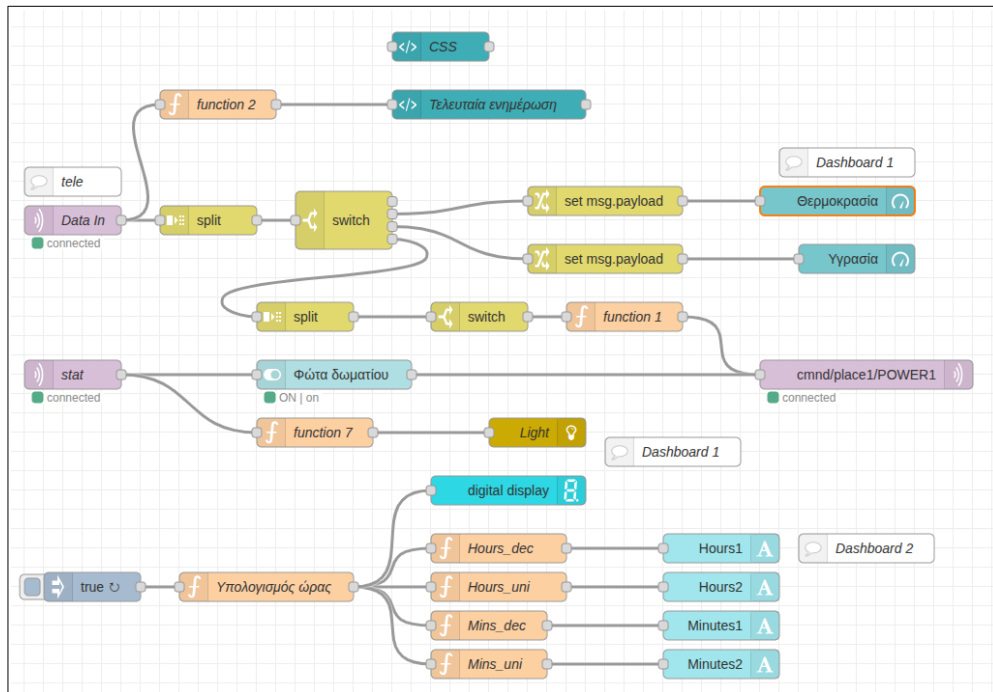
Ο κώδικας της **function 1**.

```
1 //Διάβασε μεταβλητή ορατή μόνο στο Flow.
2 //Την 1η φορά δεν έχει οριστεί, θα πάρει τιμή 'OFF'
3 let cur_state = flow.get('POWER1_STATE') || 'OFF';
4 let a = msg.payload; //Διάβασε μήνυμα από προηγούμενο κόμβο
5 if (cur_state != a) //Αν υπάρχει αλλαγή κατάστασης
6 {
7   msg.payload = a; //Να προωθήσεις την νέα κατάσταση
8   //Αποθήκευσε την νέα κατάσταση για την επόμενη φορά
9   flow.set('POWER1_STATE', a);
10  return msg;
11 }
```

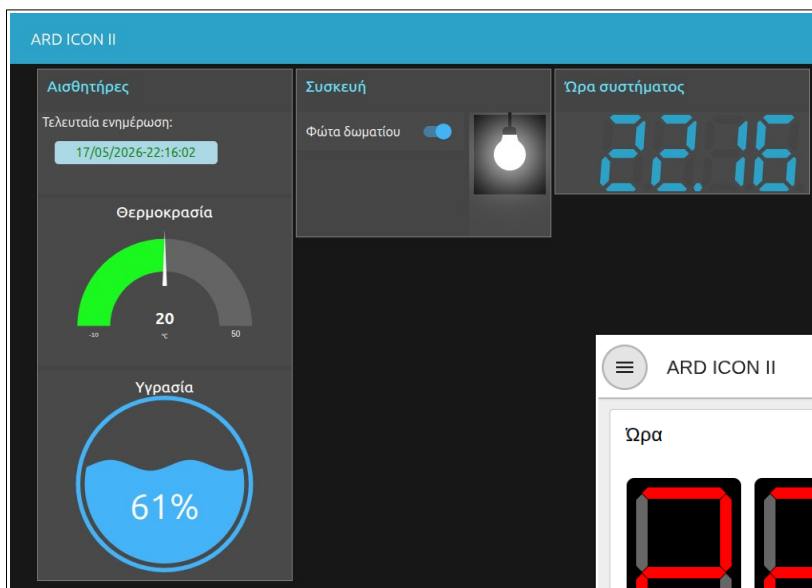
Η παραπάνω διαδικασία με τα μηνύματα **Retain** θα μπορούσε να αντιμετωπιστεί και με άλλον τρόπο, αποθηκεύοντας την κατάσταση του διακόπτη σε ένα αρχείο στο filesystem του ARD ICON II, μετά από κάθε αλλαγή θέσης.

Μέσα στον φάκελο της άσκησης υπάρχουν τα αρχεία σε μορφή **json**. Μπορείτε να τα κάνετε **import** από τις τρεις γραμμές πάνω δεξιά – Import – επιλογή αρχείου και **Deploy**. Θα πρέπει να ρυθμίσετε τις παραμέτρους του mqtt broker και το όνομα χρήστη και κωδικό γιατί δεν αποθηκεύονται για λόγους ασφαλείας.

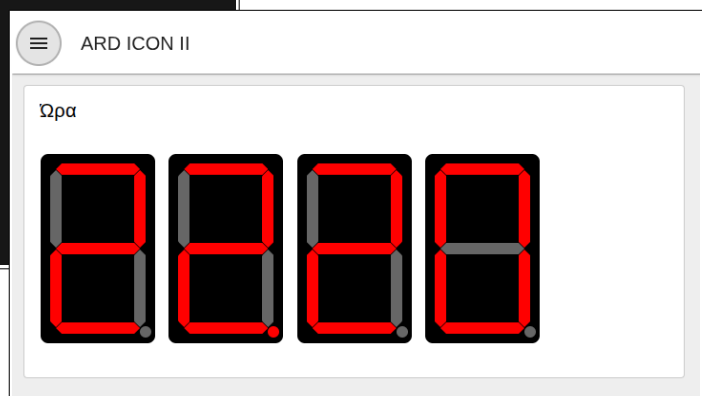
Υπάρχει και ένα άλλο flow στα παραδείγματα το οποίο κάνει χρήση του **dashboard 1**. Επειδή είναι παλαιότερο έχει πολλά έτοιμα controls τρίτων. Αυτό επιπλέον χρησιμοποιεί το node **@studiobox/node-red-contrib-ui-widget-bulb-basic** το οποίο πρέπει να εγκαταστήσετε.



Για να δούμε το **dashboard 1** γράφουμε <http://local-server-ip:1880/ui> .



Παράλληλα μπορούμε να χρησιμοποιούμε και το **dashboard 2** στην διεύθυνση <http://local-server-ip:1880/dashboard> .



Βίντεο επίδειξης στο youtube: <https://youtu.be/IIW9qPHaIHA>

Παράρτημα Α'

Εγκατάσταση και δοκιμή του Mosquitto MQTT

Το **Mosquitto MQTT broker** της **Eclipse** είναι ένας μεσίτης **mqtt** ανοιχτού λογισμικού για συστήματα **linux**, **windows** και **mac**. Αν και θα μπορούσαμε να το εγκαταστήσουμε κανονικά στο σύστημά μας, θα προτιμήσουμε την χρήση **container** για αυτονομία και εύκολη μεταφορά σε άλλα συστήματα.

Τον broker μπορούμε να τον λειτουργήσουμε αρχικά στο τοπικό μας δίκτυο LAN και μετά να το κάνουμε δημόσιο στο διαδίκτυο ώστε να είναι ορατός σε όλες τις συσκευές που ανταλλάσσουν μηνύματα με αυτόν και βρίσκονται σε απομακρυσμένες τοποθεσίες εκτός τοπικού δικτύου.

Το σχολικό δίκτυο **sch.gr** παρέχει στατικές διευθύνσεις IP και ονόματα dns σε όλα τα σχολεία της χώρας επομένως στήνοντας έναν **Linux server** ή ένα **raspberry pi** μπορούμε να χρησιμοποιούμε τον ιδιωτικό μας broker από παντού. Εναλλακτικά αν έχουμε οικιακό δίκτυο από κάποιον πάροχο μπορούμε να βάλουμε μια δωρεάν υπηρεσία όπως **dyndns** ή **freedns** ώστε να αντιστοιχίζεται η δυναμική διεύθυνση IP που αλλάζει (σύμφωνα με την πολιτική του εκάστοτε παρόχου), με ένα δημόσιο όνομα π.χ. **myserver.moool.com**. Προσοχή πρέπει να ανοίξουν κάποιες θύρες TCP στον δρομολογητή μας και συγκεκριμένα η θύρα 1883.

1. Εγκατάσταση του docker

Σε σύστημα **Ubuntu** ή **Debian** Linux ή σε ένα **Raspberry PI** θα εγκαταστήσουμε το **docker** και το **compose**. Εναλλακτικά σε σύστημα Windows 10 ή 11 θα πρέπει να εγκαταστήσουμε αρχικά το **WSL** (Windows Subsystem for Linux) και μετά να εγκαταστήσουμε το **Docker Desktop**.

Αν δεν διαθέτετε φυσικό υπολογιστή με Linux μπορείτε να δοκιμάσετε μια διανομή π.χ. Debian ή Ubuntu ως εικονικές μηχανές **VirtualBox** στον οικοδεσπότη (Host) με τα Windows. Για ευκολία στις Ρυθμίσεις – Δίκτυο επιλέγουμε Γεφυρωμένη κάρτα ώστε να λάβει διεύθυνση από τον dhcp server του τοπικού δρομολογητή και να είναι ορατή απ' όλες τις συσκευές του LAN.

Ακολουθούν οι οδηγίες για συστήματα Linux:

Η εντολές μπορούν να εκτελούνται και απομακρυσμένα μέσω SSH.

```
sudo apt install docker.io docker-compose
```

Για να τρέχει σαν κοινός χρήστης χωρίς sudo:

```
sudo usermod -aG docker $USER  
newgrp docker
```

Ελέγχουμε έκδοση με:

```
user@raspberrypi:~ $ docker --version  
Docker version 26.1.5+dfsg1, build a72d7cd
```

Δοκιμάζουμε αν λειτουργεί το docker με την εντολή `docker run hello-world`.

2. Εγκατάσταση του Mosquitto MQTT Broker

Στον κατάλογο `/opt` (`cd /opt`) φτιάχνουμε αρχείο **compose.yml** με την εντολή `sudo nano compose.yml`.

Μέσα αντιγράφουμε τα παρακάτω:

```
services:  
  mosquitto:  
    container_name: mosquitto  
    restart: unless-stopped  
    image: eclipse-mosquitto
```



```
volumes:
  - /opt/mosquitto:/mosquitto
  - /opt/mosquitto/data:/mosquitto/data
  - /opt/mosquitto/log:/mosquitto/log
ports:
  - 1883:1883
  - 9001:9001
command: 'mosquitto -c /mosquitto-no-auth.conf'
```

Αποθήκευση με **ctrl + X** και μετά **'y'** (Yes).

Στη συνέχεια μέσα από τον **opt** φτιάχνουμε κατάλογο με **sudo mkdir -p mosquitto/config**.

Μπαίνουμε στον κατάλογο με **cd mosquitto/config** και δημιουργούμε αρχείο **mosquitto.conf** με την εντολή **sudo nano mosquitto.conf**. Το αρχείο περιέχει τις ακόλουθες ρυθμίσεις:

```
persistence true
persistence_location /mosquitto/data/
log_dest file /mosquitto/log/mosquitto.log
listener 1883
allow_anonymous true
```

Γυρίζουμε πίσω στον **/opt** (**cd /opt**) και γράφουμε **docker-compose up -d**.

Κατεβάζει και εκτελεί το image.

Με **docker ps** βλέπουμε αν τρέχει το container.

```
user@raspberrypi:/opt $ docker ps
CONTAINER ID   IMAGE                COMMAND                  CREATED        STATUS        PORTS
b85be9dbdf95   eclipse-mosquitto    "/docker-entrypoint..." 43 seconds ago Up 41 seconds 0.0.0.0:1883->1883/tcp,
:::1883->1883/tcp, 0.0.0.0:9001->9001/tcp, :::9001->9001/tcp  mosquitto
```

Κάνουμε επανεκκίνηση και ελέγχουμε αν το container ξεκινάει αυτόματα.

Με την εντολή **docker images** βλέπουμε τα images που έχουν μεταφορτωθεί.

Με την εντολή **docker exec -it mosquitto /bin/sh** μπορούμε να συνδεθούμε στο **shell** του container ώστε να εντοπίσουμε πιθανά προβλήματα.

3. Δοκιμή μηνυμάτων mqtt

Αρχικά βάζουμε στο Linux ή Raspberry τα πακέτα **mosquitto-clients** με:

```
sudo apt install mosquitto-clients
```

Από ένα τερματικό κάνουμε συνδρομή σε ένα topic γράφοντας:

```
mosquitto_sub -h localhost -t test_Topic/#
```

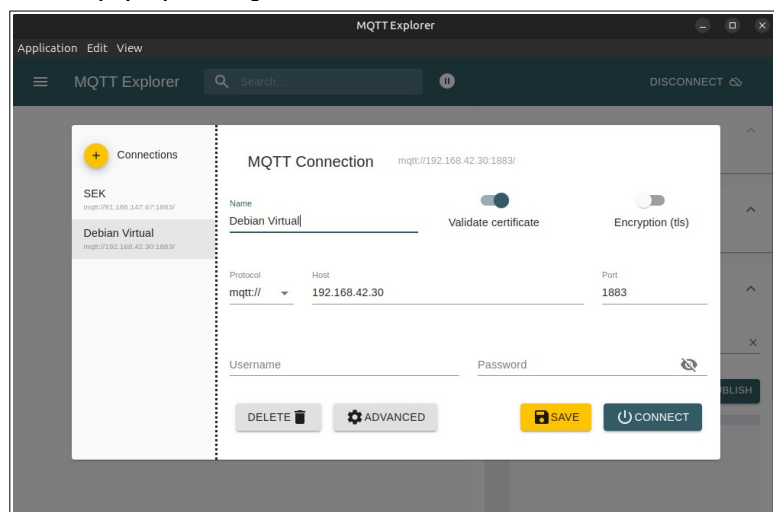
Και από ένα άλλο δημοσιεύουμε γράφοντας:

```
mosquitto_pub -h localhost -t test_Topic/tele -m temp=25.4
```

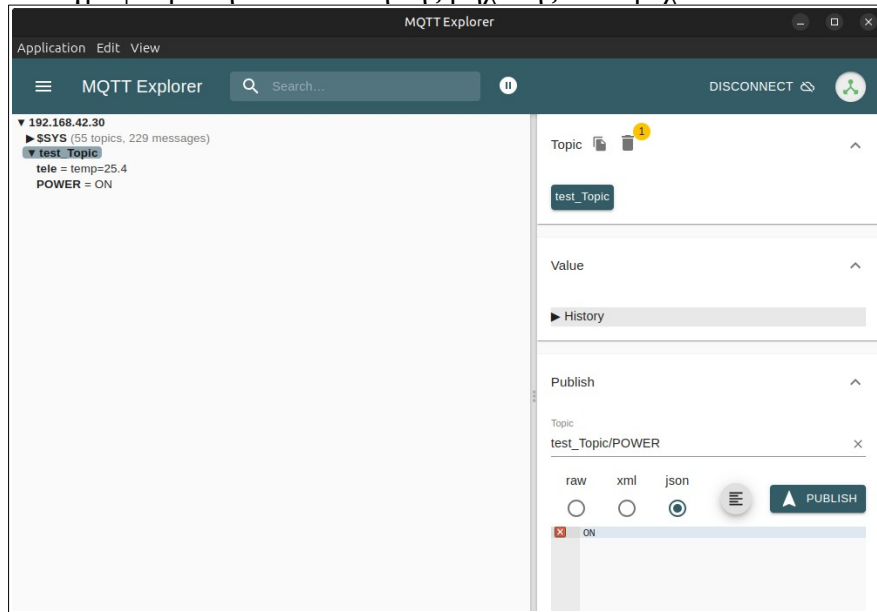
Ελέγχουμε στο πρώτο τερματικό αν έφτασε το μήνυμα **temp=25.4**.

Ένα πολύ καλό γραφικό εργαλείο ανοικτού λογισμικού για έλεγχο και εκσφαλμάτωση, είναι ο **MQTT Explorer** διαθέσιμο για Windows, Linux και Mac.

Αρχικά κάνουμε την σύνδεση από οποιονδήποτε υπολογιστή του τοπικού δικτύου.



Προσοχή στον Host γράφουμε την διεύθυνση της μηχανής που τρέχει το docker container.



Αφού συνδεθεί βλέπουμε δέντρο με τα topics και τις τρέχουσες τιμές. Στην δεξιά στήλη μπορούμε να δημοσιεύσουμε σε όποιο topic θέλουμε. Αν δεν υπάρχει θα δημιουργηθεί.

4. Πιστοποιημένη σύνδεση

Παραπάνω γινόταν ανώνυμη σύνδεση στον broker. Αν θέλουμε να χρησιμοποιηθεί username και password τότε κάνουμε τις επόμενες αλλαγές:

Αρχείο /opt/mosquitto/config/mosquitto.conf :

```
persistence true
persistence_location /mosquitto/data/
log_dest file /mosquitto/log/mosquitto.log
listener 1883
allow_anonymous false
password_file /mosquitto/data/passwd
```

Μετά στο αρχείο /opt/compose.yml τροποποιούμε την τελευταία γραμμή σε :

```
command: 'mosquitto -c /mosquitto/config/mosquitto.conf'
```

Εφαρμογή αλλαγής στο container :

```
docker-compose up -d
```

Προαιρετικά επανεκκίνηση του container :

```
docker restart mosquitto
```

Σύνδεση στο container με τερματικό :

```
docker exec -it mosquitto /bin/sh
```

Ορισμός ενός χρήστη :

```
mosquitto_passwd -c passwd mqttuser
```

Ζητάει κωδικό δύο φορές π.χ. 123456

Έξοδος από container shell με την εντολή **exit** .

5. Δοκιμή με πιστοποιημένη σύνδεση

Για να κάνω pub γράφω:

```
mosquitto_pub -h localhost -t test_Topic/tele -m temp:26.8 -u mqttuser -P 123456
```

Για να κάνω sub γράφω:

```
mosquitto_sub -h localhost -t test_Topic/# -u mqttuser -P 123456
```

Στον MQTT Explorer στην σύνδεση συμπληρώνω το Username και το Password :

The screenshot shows the MQTT Explorer interface. On the left, under 'Connections', there are two entries: 'SEK' (mqtt://81.186.147.67:1883/) and 'Debian Virtual' (mqtt://192.168.42.30:1883/), which is selected. The main panel displays the configuration for 'Debian Virtual' with the URL 'mqtt://192.168.42.30:1883/'. The configuration includes a 'Name' field with 'Debian Virtual', a 'Validate certificate' toggle (checked), and an 'Encryption (tls)' toggle (unchecked). Below these are fields for 'Protocol' (mqtt://), 'Host' (192.168.42.30), and 'Port' (1883). At the bottom, there are fields for 'Username' (mqttuser) and 'Password' (123456). At the very bottom, there are four buttons: 'DELETE' (with a trash icon), 'ADVANCED' (with a gear icon), 'SAVE' (yellow button with a save icon), and 'CONNECT' (blue button with a power icon).

Παράρτημα Β'

Εγκατάσταση του Node Red

Το περιβάλλον Node Red είναι ένα πολύ ισχυρό γραφικό περιβάλλον προγραμματισμού με blocks. Είναι φτιαγμένο σε **NodeJS** με όλα τα πλεονεκτήματα της γλώσσας όπως ταχύτητα και ασύγχρονη εκτέλεση και το υποστηρίζει μια μεγάλη κοινότητα. Μπορούμε να βρούμε πολλά έτοιμα Nodes εκτός των ενσωματωμένων όπως μηνύματα σε Viber και Telegram, αλληλογραφία, γραφικά στοιχεία για το dashboard και πολλά άλλα.

Αρχικά στο αρχείο **/opt/compose.yml** του linux host προσθέτω κάτω από το container του mosquito τα ακόλουθα:

```
node-red:
  container_name: nodered
  restart: unless-stopped
  image: nodered/node-red:latest
  environment:
    - TZ=Europe/Athens
  ports:
    - "1880:1880"
  networks:
    - node-red-net
  volumes:
    - node-red:/data

networks:
  node-red-net:

volumes:
  node-red:
```

Δημιουργώ κατάλογο **/opt/node-red** με **sudo mkdir /opt/node-red** και κατεβάζω images και δημιουργώ το container με **docker-compose up -d**.

Ελέγχω αν λειτουργεί με **docker ps**. Τώρα πρέπει να βλέπω δύο containers:

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
NAMES					
29d3a206d578	nodered/node-red:latest	"/entrypoint.sh"	10 minutes ago	Up 10 minutes (healthy)	0.0.0.0:1880->1880/tcp, :::1880->1880/tcp
nodered					
9e6323e6af1d	eclipse-mosquitto	"/docker-entrypoint..."	24 hours ago	Up 3 hours	0.0.0.0:1883->1883/tcp, :::1883->1883/tcp,
0.0.0.0:9001->9001/tcp, :::9001->9001/tcp	mosquitto				

Από browser συνδέομαι στην διεύθυνση **http://διεύθυνση_linux_host:1880** π.χ. <http://192.168.42.30:1880>. Εκεί βλέπω το περιβάλλον προγραμματισμού του Node Red.

Αντιγράφω τον κατάλογο **data** του container στο **/opt/node-red** με **sudo docker cp nodered:/data /opt/node-red/**. Τώρα μέσα στο **/opt/node-red** δημιουργήθηκε και ο κατάλογος **data** με όλα τα δεδομένα του container.

Αλλάζω το αρχείο **/opt/compose.yml** στην ενότητα του node-red ως εξής:

```
node-red:
  container_name: nodered
  restart: unless-stopped
  image: nodered/node-red:latest
  environment:
    - TZ=Europe/Athens
  ports:
    - "1880:1880"
  volumes:
    - /opt/node-red/data:/data
```

Σταματάω το container με `sudo docker stop nodered`.

Εκτελώ πάλι την compose με `docker-compose up -d`.

Αλλάζω ιδιοκτητή στον data με `sudo chown -R 1000:1000 /opt/node-red/data`.

Κάνω εκτελέσιμα τα αρχεία το nodered μέσα στο data με `sudo chmod +x -R /opt/node-red/data/`.

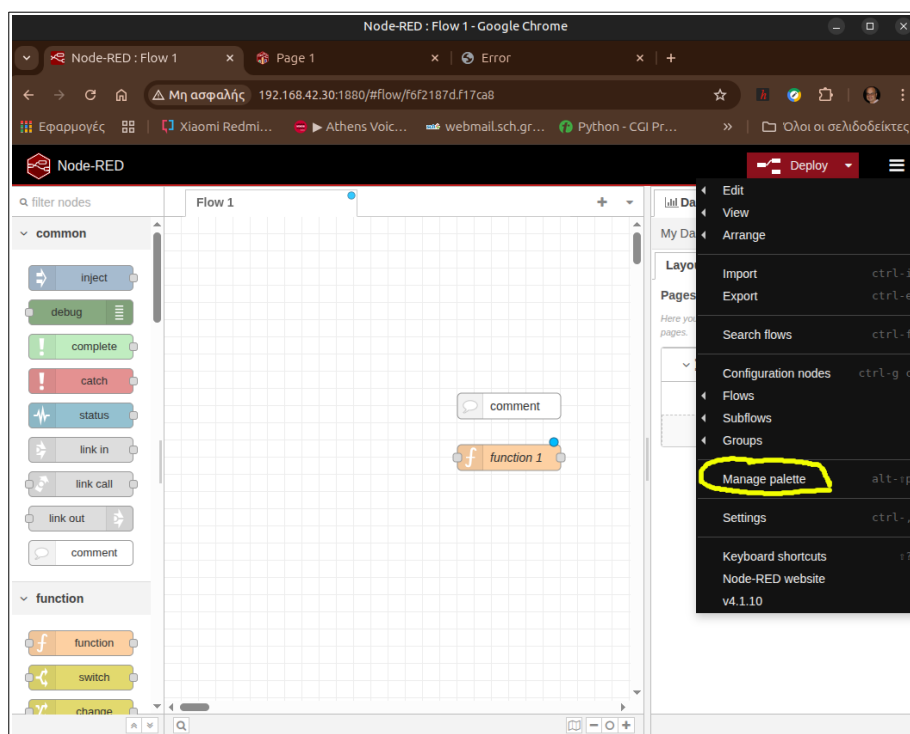
Τέλος δοκιμάζω αν τα δεδομένα γράφονται και αποθηκεύονται στον κατάλογο /opt/node-red/data εκτός του container. Γι' αυτό βάζω μερικά nodes μέσα στο flow και πατάω Deploy.

Επανεκκινώ το container με `sudo docker restart nodered` και επαναφορτώνω την σελίδα στο browser και ελέγχω αν οι αλλαγές υπάρχουν και δεν φορτώθηκε η σελίδα υποδοχής του nodered.

Για σύνδεση με το shell του container γράφω: `sudo docker exec -it nodered /bin/bash`

Προσθήκη επιπλέον nodes

Αρχικά θα βάλουμε το γραφικό περιβάλλον χρήστη ui που είναι το **dashboard2**. Πατάμε πάνω δεξιά στις τρεις γραμμές και μετά **Manage palette**. Πατάμε στο tab **Install** και αναζητούμε **@flowfuse/node-red-dashboard** και πατάμε του κουμπί install. Στο μήνυμα για τις εξαρτήσεις απαντάμε καταφατικά με install και μετά επανεκκινούμε το nodered από τερματικό με `sudo docker restart nodered`.



Τώρα στα εργαλεία αριστερά έχει προστεθεί μια νέα ομάδα στο τέλος με διακόπτες, sliders, labels κλπ.

Αν θέλουμε μπορούμε να εγκαταστήσουμε και το παρωχημένο **dashboard** αναζητώντας το ως **node-red-dashboard** όπου θα προσθέσει μια νέα ομάδα.

Ο τελικός χρήστης βλέπει το παλιό dashboard με <http://192.168.42.30:1880/ui> και το dashboard 2 δίνοντας την διεύθυνση <http://192.168.42.30:1880/dashboard> .

Για να φτιάξουμε τα παραδείγματα εγκαθιστούμε για το παλιό dashboard 1 τα **node-red-contrib-ui-digital-display** και **node-red-contrib-ui-level**. Επίσης για το νέο dashboard 2 βάζουμε το **@yoshoku/node-red-dashboard-2-ui-seven-segment-display** .

